City St George's, University of London

MSc in Artificial Intelligence

2026

Token Budget Orchestration Strategies for Web Agents

Thomas Beard

Student Number: 220008104

Supervisor: Professor Eduardo Alonso

Declaration

By submitting this work, I declare that this work is entirely my own except those parts duly identified and referenced in my submission. It complies with any specified word limits and the requirements and regulations detailed in the assessment instructions and any other relevant programme and module documentation. In submitting this work, I acknowledge that I have read and understood the regulations and code regarding academic misconduct, including that relating to plagiarism, as specified in the Programme Handbook. I also acknowledge that this work will be subject to a variety of checks for academic misconduct.

***Signed**: Thomas Beard*

October 14th, 2026

Abstract

This project investigates whether multi-agent orchestration of web agents still pays when the token budget is fixed. A single web agent (A) was compared with a fixed Planner–Executor–Critic pipeline (B) and with a router trained by offline reinforcement learning (C), which chooses between A's and B's mode of operation once per episode and may stop an episode early, on 475 WebArena tasks at six token budgets from 2,000 to 64,000 tokens (9,450 episodes, 116 million tokens). A pipeline step was found to cost 2.8 times a single-agent step, and B needed two to four times A's budget to reach the same success; its Critic consumed 31.5% of B's spend while revising only 5.1% of actions. The router kept 93% of A's successes while spending 20% fewer tokens, mainly by declining tasks it predicted hopeless, and no within-episode allocation scheme changed any strategy's success rate. Under a fixed budget, orchestration therefore must earn its overhead, and the useful, learned decision was whether to attempt a task rather than how to orchestrate it.

Keywords: web agents; WebArena; multi-agent orchestration; token budgets; offline reinforcement learning.

Table of Contents

Token Budget Orchestration Strategies for Web Agents 1
Declaration 2
Abstract 3
1 Introduction and Objectives 7
1.1 Background and Motivation 7
1.2 Problem statement and research questions 7
1.3 Objectives and success tests 8
1.4 Beneficiaries 8
1.5 Work Plan 9
1.6 Change of goals and methods during the project 9
1.7 Structure of the report 10
2 Context 11
2.1 Web agents and the WebArena benchmark 11
2.2 Multi-agent orchestration and the fixed pipeline 11
2.3 Token cost as a structural constraint 12
2.4 Budget aware orchestration and learned routing 13
2.5 Evaluation practice in agent research 13
2.6 Summary and the gap addressed 14
3 Methods 15
3.1 Experimental design 15
3.2 Environment and infrastructure 15
3.3 Token accounting and budget enforcement 16
3.4 Strategy implementations 17
3.5 Data collection, logging and task classification 20
3.6 Verification and validation 24
3.7 Ethics, legal, social and professional issues 25
4 Results 25
4.1 What was produced 26
4.2 The collected dataset 26
4.3 RQ1 – Success and cost against token budget 27
4.4 RQ2 – Sensitivity to task difficulty 31
4.5 Critic behaviour in Strategy B 33
4.6 RQ3 – Intra-episode allocation 33
4.7 Summary of results 35
5 Use of Generative AI 36

5.1 Declaration of use 36
5.2 How AI was used in code development 36
5.3 How outputs were evaluated before integration 37
5.4 Where AI was wrong 37
5.5 Reflection 37
6 Discussion 38
6.1 The cost of orchestration 38
6.2 What review is worth in a fixed pipeline 38
6.3 Learned routing against the alternatives 38
6.4 Confidence, validity and generalisability 39
6.5 Implications and recommendations 39
7 Evaluation, Reflections and Conclusions 40
7.1 Evaluations against the objectives 40
7.2 Evaluation of the methods and the plan 40
7.3 Reflections on the project process 40
7.4 Contributions 40
7.5 Further work 41
7.6 Conclusions 41
Glossary 42
References 44
Appendix A – Full Results Tables 47
A.1 Strategy A – Complete per-budget, per-site, and per-tier breakdowns 47
A.2 Strategy B – Complete per-budget, per-site, and per-tier breakdowns 48
A.3 Strategy C - Complete per-budget, per-site, and per-tier breakdowns 50
A.4 Strategy Comparisons - Strategy comparisons: paired tests, equivalent budgets and failure modes 51
A.5 Spending schemes 56
Appendix B – Source Code 60
B.1 Repository 60
B.2 Repository structure and file inventory 60
B.3 Installation and reproduction guide 60
B.4 Code Listing 61
B.5 Agent prompts and output schemas 62
Appendix C – Dataset 68
C.1 Episode JSONL Schema 68
C.2 Datasets inventory 68

C.3 Task classification listing 71
C.4 Spending-Scheme configuration 71
C.5 Access/Location 72
Appendix D – Generative AI Use 73
D.1 Code generation – Prompt log 73
Appendix E – Project Management and Ethics 74
E.1 Research Ethics Review Form (CSREC) 74
E.2 Work plan: planned against actual 77
E.3 Risk register review 77
E.4 Record of supervisory contact 80
E.5 Project Journal 80
Appendix F – Original Proposal 81

1 Introduction and Objectives

1.1 Background and Motivation

A large language model (LLM) agent today can go and interact with real websites. These websites are typically interacted with by specifying a plain language objective to a web agent. The web agent can interpret a web page, choose a thing to do on the page such as a click or filling out a web form, observe the results of the chosen action, and repeat until the goal of the specified task is complete or a termination criterion is met. This type of goal solving can be made much better by adding multi-agent orchestration. Typically, such an orchestration system consists of three LM's: a planner that decomposes the specified goal into sub-goals, an LM that executes the goals found by the planner, and an LM that after each action by the executor criticizes the action just taken by the agent.

How many tasks can be done with a fixed number of tokens? And will results vary with a budget? These are essentially the questions that can now be answered for various architectures for web interaction using WebArena (Zhou et al., 2023), a testing framework for web interaction by large language model (LLM) agents, which contains many functional websites that can be used to test a LLM agent. There is sufficient variability in the difficulty of the websites that the authors report that while a GPT-4 LLM agent succeeded in 14% of the tasks on a website, human interaction succeeded in 78% of the same tasks.

To explore these trade-offs, I will look to add evaluation of various orchestration strategies of agentic LLMs to the existing WebArena (Zhou et al., 2023) framework, running them under fixed budgets of tokens rather than optimizing for peak performance. WebArena is a set of functional mockups of real websites, with programmatic end-to-end tasks, which achieve roughly 14% success with a GPT-4 LLM vs. 78% with a human, on tasks that are challenging enough to be worth simulation. These are three areas that I would like to become more deeply embedded in: agentic LLMs; offline reinforcement learning; and controlled empirical evaluation.

1.2 Problem statement and research questions

The problem addressed is that the token cost of multi-agent orchestration for web agents is not understood as a function of budget as there is no controlled evidence on when a fixed pipeline justifies its overhead, nor on whether the decision to orchestrate can be learned per task rather than fixed in advance. The project compares three orchestration strategies under identical, fixed per-episode token budgets.

Strategy A, a single agent that selects every action itself; Strategy B, a fixed Planner, Executor, Critic pipeline that pays all three roles on every step; and Strategy C, a dynamically routed system in which a router trained with offline reinforcement learning chooses per episode between A's and B's mode of operation and may stop an episode early when the remaining budget is unlikely to be converted into a success.

Three research questions, carried unchanged from the proposal, structure the work:

RQ1: How does task success rate, token efficiency and error profile vary as a function of a fixed global token budget when comparing the three orchestration strategies on WebArena?
RQ2: How do the three strategies differ in sensitivity to task difficulty (Easy / Medium / Hard) in terms of success rate and failure modes across WebArena tasks?

RQ3: Given the same global token budget, how do different intra-episode allocation schemes (front-loaded, even-distribution, reactive) affect success rate and token efficiency across all three strategies?

1.3 Objectives and success tests

The proposal's five objectives, the results of which are discussed in Chapter 7.

Objective 1 - Implement all three strategies mapped to the BrowserGym/WebArena environment with a simple token accounting wrapper that has a fixed budget for each episode. Test: all three strategies run end to end on the self-hosted WebArena sites, and every episode is recorded with the same various fields such as (but not limited to): task goal, step log, cumulative tokens, action error, etc.
Objective 2 - Collect a labelled dataset of WebArena tasks for different budget sizes. This will be used as both a test set for the strategies and a training set for Strategy C's router. Test: every (task, budget) cell is covered for A and B; every episode carries a difficulty tier, a risk level and a reward-eligibility flag; the LLM-assigned tiers are checked against 50 hand-labelled tasks, with agreement measured on tasks the annotator never saw.
Objective 3 - Train and evaluate a lightweight MLP router using offline reinforcement learning with entropy regularisation. Test: in cross-validated offline replay the router keeps at least 90% of Strategy A's successes, and is then run live as Strategy C across all budgets.
Objective 4 - Produce empirical cost-performance curves and conduct the necessary difficulty-stratified analysis of the three researched hypotheses. Test: All assertions about differences between strategies must be verified by paired statistical tests (in this case exact McNemar tests on tasks executed by both strategies) and, where appropriate, corrected for multiple testing. Success rates are then reported.
Objective 5 - Document all findings within this dissertation while ensuring that code, datasets, and analytical scripts are systematically organized for reproducibility. Test: Develop a repository that covers all modules (collection, decontamination, training & analysis) in detail and has proper documentation for every module. The schema and locations for all datasets produced will be laid out in the appendices.

1.4 Beneficiaries

This study functions as a controlled comparative study that will be of interest to other work that is budget-sensitive and multi-agent in nature. The full multi-episode annotated dataset is provided that includes categorization by primary difficulty tier and failure mode(s) for each. Further, a decontamination procedure for the provided benchmarks is supplied that preserves the site state across episodes. Of interest to the benchmark and evaluation community, this study serves as a primary experimental control where cost has been held constant as opposed to simply being reported as a secondary metric. Thus, organizations that are comparing the cost of an individual automated agent to the costs of the overheads induced by orchestration of such

agents can gather sufficient evidence to determine whether or not such overheads are justifiable.

1.5 Work Plan

Figure *1.1* shows the work plan as a breakdown, with the phases ordered left to right. The project moved through four milestones; a working environment (self-hosted WebArena on AWS with the token accounting wrapper); a complete, decontaminated A & B dataset across the six-budget ladder; a trained and deployed C router and finally the analysis and write up. Task preparation (covering the difficulty annotation validated against a hand labelled set and the contamination risk classification) sits before collection because every later phase uses its labels. The A & B batches serve twice, as the RQ1 evaluation data and as the routers training data, so C's branch follows the main collection path, and the RQ3 scheme runs reuse the finished strategies on a read only task sample. The dates on which each phase was completed are in Appendix E.2

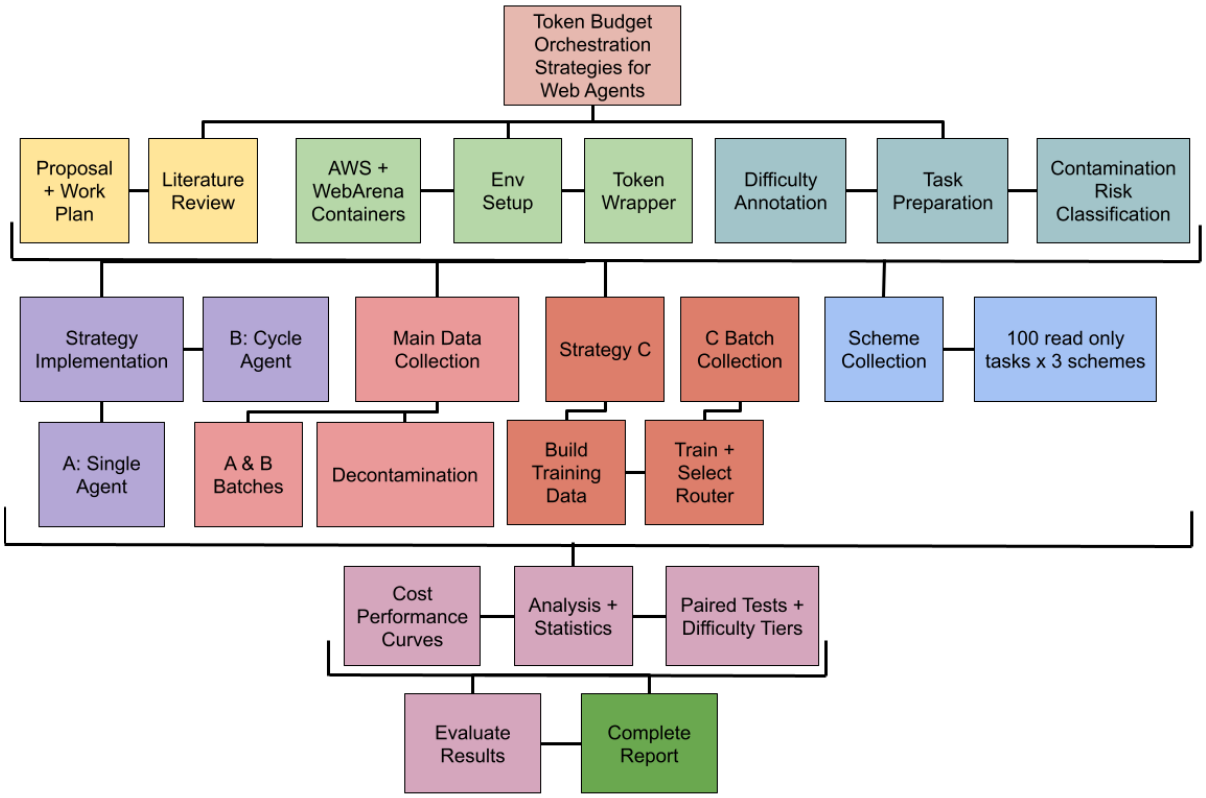


Figure 1.1 Work Plan

1.6 Change of goals and methods during the project

The research questions of this study were consistently addressed throughout the study. Nevertheless, the methodologies for addressing the research questions were revised as the research progressed.

Budget Ladder: Originally, I planned to assess four different budget levels of 2,000 tokens, 8,000 tokens, 16,000 tokens, and 32,000 tokens. Since Strategy B executions rarely completed tasks for fewer than 16,000 tokens, I increased the number of budget levels to six (2,000 to 64,000 tokens).

Task Sample: I restricted evaluation to 475 single-site tasks distributed over 3 of the 5 platforms in WebArena, i.e., shopping, shopping admin, and the forum. All multi-site tasks were excluded from our study to keep the cost of collecting enough data within bounds and to be able to assess the self-hosted containers for extended periods of time.

Decontamination: because the sites keep state between runs, a success on a state-changing task could make later repeats trivial. A risk classification and a first-success rule were added: episodes after the first success on such a task are excluded from reward and analysis, and RQ3 used a stratified sample of 100 read-only tasks for the same reason.

Strategy C Design: the proposal had the router choosing which role to call at each step. This became a two-head design: a mode head that chooses once per episode between A's single-agent mode and B's full pipeline, and a stop head that decides at each step whether to continue. The offline setting forced the change: logged A and B episodes only ever call the roles in those two patterns, so a per-step role policy had nothing to learn from, whereas the episode-level choice is supported by paired episodes on the same tasks.

1.7 Structure of the report

Chapter 2 reviews the context: web agents and WebArena, multi-agent orchestration, token cost as a constraint, budget-aware and learned routing, and evaluation practice, ending with the gap this project addresses.

Chapter 3 describes the methods: the experimental design, infrastructure, token accounting, the three strategy implementations, data collection and decontamination, difficulty annotation, and verification, closing with ethical and professional issues.

Chapter 4 Chapter 4 presents the results by research question.

Chapter 5 declares and reflects on the use of generative AI.

Chapter 6 discusses the findings, their validity, and their implications.

Chapter 7 evaluates the project against the objectives in 1.3, reflections on the process, the limitations, and finally setting out contributions and further work.

2 Context

This chapter reviews the state of knowledge the project builds on: web agents and the benchmark they are evaluated on (2.1), the multi-agent patterns whose cost is in question (2.2), what is known about that cost (2.3), existing budget-aware and learned approaches (2.4), and how agent research evaluates its claims (2.5).

2.1 Web agents and the WebArena benchmark

Research on web agents, defined as systems that can manipulate a website's interface to achieve goals specified in natural language, was already conducted before the LLMs became available. Shi et al. (2017) in their work on World of Bits first presented web interaction as a reinforcement learning problem and afterwards in their experiments conducted behavioural cloning and fine-tuned policy by policy gradient. They reached the conclusion that even after fine-tuning their agents performed way worse than humans. An important conclusion of their research is that the web is a very natural domain for agent research because it is very realistic and there are a lot of tasks available. LLMs reopened the field by replacing learned low-level control with language conditioned action selection, and attention shifted to benchmarks realistic enough to support claims about practical use.

The WebArena (Zhou et al., 2023) benchmark is used for this project. WebArena consists of fully functional web sites: e-commerce sites, business management site, and a site like Reddit for discussion forums. Each of the web sites is populated with real data, and are all contained in Docker containers so that fully functional web sites can be self-hosted. There are 812 tasks for WebArena. All these tasks are expressed as natural language intentions. Two major design decisions were made by the authors of the benchmark to enable a budget-controlled study. First, the sites can be self-hosted. This means that the sites can be replayed in a given state to assess the behaviour of a given program for a certain number of steps and then rolled back to the starting state to test the behaviour of another program. In this way, a given number of steps can be taken to assess a program for different budgets. Second, the goal of the benchmark is to assess functional correctness of a web site. In other words, did the web site end up in the correct state. The study found that the best GPT-4 agent was able to complete 14.41% of the tasks, whereas a human completed 78.24% of the same tasks. The main reasons for failure of the GPT-4 agents to complete tasks were lack of active exploration and failure to recover from errors.

This project is using BrowserGym (Le Sellier De Chezelles et al., 2024) to access the tasks and experiments in WebArena and the other benchmarks that are part of the gym, modelling partial observability and structured HTML pages as Accessibility Trees with ids for all elements.

2.2 Multi-agent orchestration and the fixed pipeline

Dividing a task into multiple roles is the prevalent way to deal with the limitations of single agents. Supporting short-horizon benchmarks as a special case for Mixture-of-Agents (MoA) is addressed in Wang et al. (2024a). In the Mixture-of-Agents framework (MoA), a first layer of so-called proposer models generates input for an aggregator in subsequent layers. The aggregator adds value beyond selecting the best input for the subsequent layers. An ensemble of currently available open-source models reaches 65.1% on AlpacaEval 2.0. In addition to the full Mixture-of-Agents framework, Wang et al. (2024a) also describe a more lightweight version MoA-Lite.

Rather than having multiple roles drafting in parallel, there are several approaches to assigning specific roles to planning, acting and evaluating. Yang et al. (2025) in Fig. 11 describe feedback topology for generating-and-critiquing structures in the form of structured pseudo-code for Planner, ToolCaller and Replanner. For example, for the simple case of Planner, ToolCaller and Replanner, this approach not only improves accuracy but also reduces the number of input tokens in some cases. For example, in terms of the number of input tokens, the improved approach of Zhang et al. (2024) in Fig. 12 to inter-agent communication outperforms the minimal approach to chain in terms of accuracy while increasing token counts from two to eleven times.

These findings are relevant to short-horizon tasks like question answering, arithmetic problems and coding challenges. In web-agents a large portion of the current page is reobserved at every step. Therefore, the increase in observation costs of a three-role architecture to three times that of a single role is compounded by an increase in the number of instructions to be issued at every step. Importantly, the cost of a number of steps into a conversation does not remain constant but increases linearly with episode length. RQ1, therefore, asks whether the increased complexity of structure over single role agents delivers value within a fixed budget.

2.3 Token cost as a structural constraint

Salim et al. (2026) in their analysis of token allocation within an operational multi-agent framework studied communication in the software development process of ChatDev. The analysis showed that the largest part of the tokens were spent in the code-review phase with an average of 59.4% of all tokens. The re-sent of context to the agents even used more tokens than necessary with an average of 53.9% of all tokens. These token-expenditures are even greater than the token-expenditure for producing artifacts. Chen et al. (2026) try to explain these results from an economic point of view. They state that the costs of inter-agent communication represent a transaction cost. Thus, a cost-efficient allocation of tokens in a multi-agent system could be very effective.

In similar research, Zhang et al. (2024) modeled communication in a multi-agent system as a message-passing graph. They created a compact sub-graph that performed similarly to the original system and then reduced the number of tokens in the communication by 28.1% to 72.8% on different host frameworks. Thus, the costs that are incurred by communication are structural and are a property of both the communication graph and the re-sent context, which are controllable rather than incidental.

Much work has been done to cut costs after a system has been designed by pruning or even compressing existing systems to make them work within a given budget. However, there is very little work done to restrict communication costs during execution with strict budgets and even less work done studying the different architectural choices to reach similar goals with given budgets. In this project we study budgetary restrictions as the independent variable and architectural choices as the dependent variable.

2.4 Budget aware orchestration and learned routing

Budget-aware multi-task architecture search (BAMAS, Yang, L. et al., 2025) tackles the problem of provisioning a set of large language models (LLMs) within a fixed budget using integer linear programming and offline reinforcement learning to search for an optimal collaboration topology for each task. BAMAS achieves huge cost reductions (up to 86%) while maintaining comparable accuracy on GSM8K, MBPP, MATH benchmarks. Importantly, we observe two behavioural properties of learned policies in such strict budget settings: (i) risk aversion, where a simple topology without a critic is preferred over a more complex one; (ii) unusual reluctance to choose a planner-driven topology, even if it is the most complex one, as the returns it can bring are far below its cost and instability.

AgentBalance (Cai, Ning and Liu, 2025) works within a set of pre-defined costs for tokens and latency budget. By first choosing an agent's backbone it then proceeds to choose a topology to achieve specified performance gains of up to 10% on a given budget when compared to alternate configurations. Most relevantly, BAVT (Li et al., 2026) allows for budget-aware control within an episodic framework by modifying a standard tree-search policy so that as a budget is depleted the policy transitions from a phase of exploration to a phase of exploitation. Consequently, it achieves greater than baseline performance when operating with a limited budget and only a small number of resources (four times as many) are allocated in a more than normally distributed, i.e. 'brute force' fashion.

Strategy C makes routing decisions for Strategy C analogous to BAMAS, using offline learning from logged episodes of Strategies A and B to select between alternatives that have been logged. This work makes novel use of many logged episodes of web interactions within a realistic operational environment limited by a strict per-episode budget, as opposed to simple mathematical tasks or question answering. For each mode A/B in Strategy C, there is also a decision at each step whether or not to expend remaining resources for that step, analogous to the use of depletion-sensitive control in BAVT.

2.5 Evaluation practice in agent research

A two-dimensional taxonomy for the evaluation of LLMs is proposed by Mohammadi et al. (2025). The paper reports that instead of being a controlled variable for the comparison, cost is reported by the authors for several aspects and methodologies to evaluate LLMs. In a re-evaluation of seven reasoning strategies for seven identical tasks after equalizing their respective computational resources, Wang et al. (2024b) found that the simple chain-of-thought strategy even outperformed more complex ones such as multi-agent debate and Reflexion, in some cases, when using self-consistency. Some of the previously reported advantages, however, were only achieved with more computational budget, while other benefits even required an increase in the overall budget. In conclusion, the authors find that tokens are the most comprehensive measure for the respective budget, as they already include the corresponding latency.

These comparative studies on Chapter 4 methods were conducted within the fixed global budget framework using a budget ladder as the only varying factor, and paired tests on identical tasks as opposed to independent tests. A special note on WebArena. Due to the persistent state between episodes on different sites, subsequent attempts to complete a task could 'inherit'

positive or negative effects from previous attempts. This issue was tackled in Section 3.5.2 by implementing a decontamination pipeline.

2.6 Summary and the gap addressed

Each of the above research strands provides essential components for the current project. However, each also has significant gaps or voids that the current project addresses. For example, there are already many realistic and reproducible web benchmarks (2.1), but few of them have clear success criteria that can be used to objectively measure the performance of a Web agent. While there is evidence that the use of orchestration patterns can improve quality for short time horizons (2.2), the cost of tokens in terms of structural cost and even redundancy has not yet been used as a significant measure for evaluation of Web agents (2.3). There are many emerging methods and tools for budget-aware construction and routing (2.4), but the current methods are mostly focused on design-time methods and even coding practices that enforce a hard budget at design time, and sometimes even use logged web episodes' data to complete design time tasks. However, the need for budget-aware evaluation has already been advocated and proven to change the conclusions drawn from previous work (2.5). However, the primary use of budget-aware evaluation as experimental control for Web agents has not yet been explored. Also, its amalgamation with paired statistics for comparing different architectures of Web agents using a stateful benchmark for evaluation has not yet been decontaminated. The current research thus targets an interesting intersection of several of the above research inquiries. RQ1 compares different architectures for a single agent using a fixed budget. RQ2 stratifies RQ1 by task difficulty. RQ3 explores the allocation of budget within an episode in more detail. The methods used in this investigation are presented in Chapter 3.

3 Methods

This chapter reports what was done, in the order it was done: the experimental design (3.1), the environment (3.2), token accounting and budget enforcement (3.3), the three strategies as implemented (3.4), the collection runs and the classification, decontamination and difficulty annotation that make the data analysable (3.5), verification and the statistical treatment (3.6), and the ethical and professional issues (3.7). Software is cited where it appears and listed with versions in Appendix B.3.

3.1 Experimental design

The study is a controlled comparison with two fully crossed factors: orchestration strategy (A, B, C) and global token budget (2k, 4k, 8k, 16k, 32k and 64k tokens per episode). Every strategy ran every task at every budget, 475 × 6 = 2,850 episodes each, so all comparisons are made within identical (task, budget) cells and are amenable to paired McNemar tests (Section 3.6). The dependent variables for RQ1 are functional success as judged by WebArena's evaluators, tokens spent and the reason each episode ended (Section 3.3).

RQ2 adds a non-manipulated factor, task difficulty (Easy, Medium or Hard; Section 3.5.3), and re-analyses the main grid by tier. RQ3 is a separate study at the 32k budget on a fixed sample of 100 read-only tasks: each strategy runs under three allocation schemes, and the main grid's 32k episodes on the same tasks serve as the pay-as-you-go reference.

The task population is the 475 single-site WebArena tasks on the shopping, shopping-administration and forum sites (Table C.3.1); multi-site tasks and the GitLab and map sites were excluded because they cannot run on one instance (Section 3.2). The RQ3 sample is restricted to read-only tasks, which repeated attempts cannot contaminate (Section 3.5.1), stratified by tier (33/33/34) and drawn with a fixed seed. Because the forum has only 11 read-only tasks, the sample is dominated by the two shopping sites (57, 39 and 4 tasks), a limitation returned to in Chapter 6.

3.2 Environment and infrastructure

The websites of WebArena are running as Docker containers (Docker Inc., 2025) on the public machine image provided by its maintainers. In accordance with the maintainer’s guidelines, the environment was deployed on 10 July on a single Amazon EC2 t3a.xlarge instance (4 vCPU, 16 GiB RAM, 1,000 GiB root volume; Amazon Web Services, 2025). Only the three relevant containers were started (shopping on port 7770, shopping administration on port 7780, forum on port 9999). The base URL of Magento was set to localhost since all agents run on the same instance. Long running sessions were executed within tmux to deal with SSH connection disruptions.

To let the agents interact with the websites within the tasks, BrowserGym 0.14.2 (Le Sellier De Chezelles et al., 2024) is used to present the tasks to the agents within gym environments. A BrowserGym environment is reset() with the start page of a website within a task and step(action) is called with a specific action. The subsequent observation and the related reward for this action is then provided by WebArena’s evaluator. The observation is the accessibility tree of a webpage which is transformed into text format and filtered for visible elements with

ids. Thus, the action click('a31') for example refers to an element that has been shown to the agent before. The actions that are used within the BrowserGym high-level set are sent to the user (e.g. for submitting a response to a task within the web environment using send_msg_to_user).

The agents were built with Pydantic AI 1.5 (Pydantic, 2025), which enforces a typed output schema on every model reply. Every agent, whatever the strategy, uses OpenAI's GPT-5 mini (OpenAI, 2025) at the lowest reasoning effort, with hidden reasoning tokens charged as output tokens. WebArena's fuzzy-match evaluator for free-text answers (WebArena, 2024) was moved from the deprecated gpt-4-1106-preview model to GPT-4o mini (OpenAI, 2024). This and the other deviations from the stock configuration, including extending BrowserGym's per-action timeout from 500ms to 10s after a click was reported as failed when it had succeeded, are documented for reproduction in Appendix B.3. Decontamination, router training (PyTorch 2.13; Paszke et al., 2019) and analysis (NumPy, Harris et al., 2020; pandas, McKinney, 2010; SciPy, Virtanen et al., 2020; Matplotlib, Hunter, 2007) ran locally under Python 3.11.

3.3 Token accounting and budget enforcement

To estimate the total number of tokens of a model call (input tokens + output tokens, all reasoning tokens are counted complete), the API documentation of the model call is queried. The budget B is restricted on two occasions: 1. Before each single step of a model call, an estimation is made of the costs that will be incurred by this step (Equation 3.1). If the estimated costs of a step would exceed the allocated budget for this step, the step is not executed, and the episode is concluded with the reason budget_would_exceed. 2. After each model call the total costs of the ongoing episode so far are checked against a cap established for the safety tokens (safety_token_cap). If the cap has been reached with the costs of the currently ongoing episode, this episode is concluded.

$$\hat{c}_t = k\left\lceil \frac{|x_t|}{4} \right\rceil + m_{\text{out}} + m_{\text{instr}} \qquad \text{proceed only if } C_{t-1} + \hat{c}_t \le B \qquad (\mathbf{3.1})$$

where $|x_t|$ is the prompt length in characters, k the number of model calls in the step, C_{t-1} the total already spent, and m_{out} and m_{instr} reserves for the models' outputs and for the instruction blocks that precede every prompt. Strategy A uses k = 1 with no reserves; Strategy B uses k = 3, m_{out} = 900 and m_{instr} of about 2,050 tokens (its three instruction blocks); strategy C uses whichever applies to the mode it has chosen. Because A's estimate covers only the prompt, its actual cost can exceed the estimate by the final step's output and instruction tokens, so the post-call check then ends the episode, which bounds any overshoot of the stated budget to one step. The size of that overshoot is reported in Chapter 4.

Three guards stop an episode from spending its budget on looping (Table C.1.1): a ceiling of 25 steps, four consecutive failed actions, and the same action issued from the same page three times. That table also gives the failure classes used for RQ1 and RQ2.

RQ3 was managed within the same budgetary constraints. Before each step of the scheme the scheme runner calculated a token allowance and a reasoning effort for that step based on the current scheme, step index and previous step outcomes. The page text in the prompt was then truncated to approximately that estimated cost for that step. As before the goal, URL, error line and history were never truncated. All allowances were calculated from the same basis: B/5 =

6,400 tokens at 32k. Hence a scheme has the same implications for all the strategies outlined in Table C.4.1. Elevated effort for Agent A and Planner B. An increased output reserve in accordance with Equation 3.1 set to 1,500 tokens for 'high effort' calls.

3.4 Strategy implementations

All three strategies share the harness of Section 3.2: the same observation, action set, prompt skeleton (goal, current URL, last action error, the last eight actions with the URLs they led to, and the page), model, guards and budget checks, all defined once in wa_env.py. They differ only in how many model calls a step makes and in who decides the action (Figure 3.1); the full text of every prompt and output schema is reproduced in Appendix B.5.

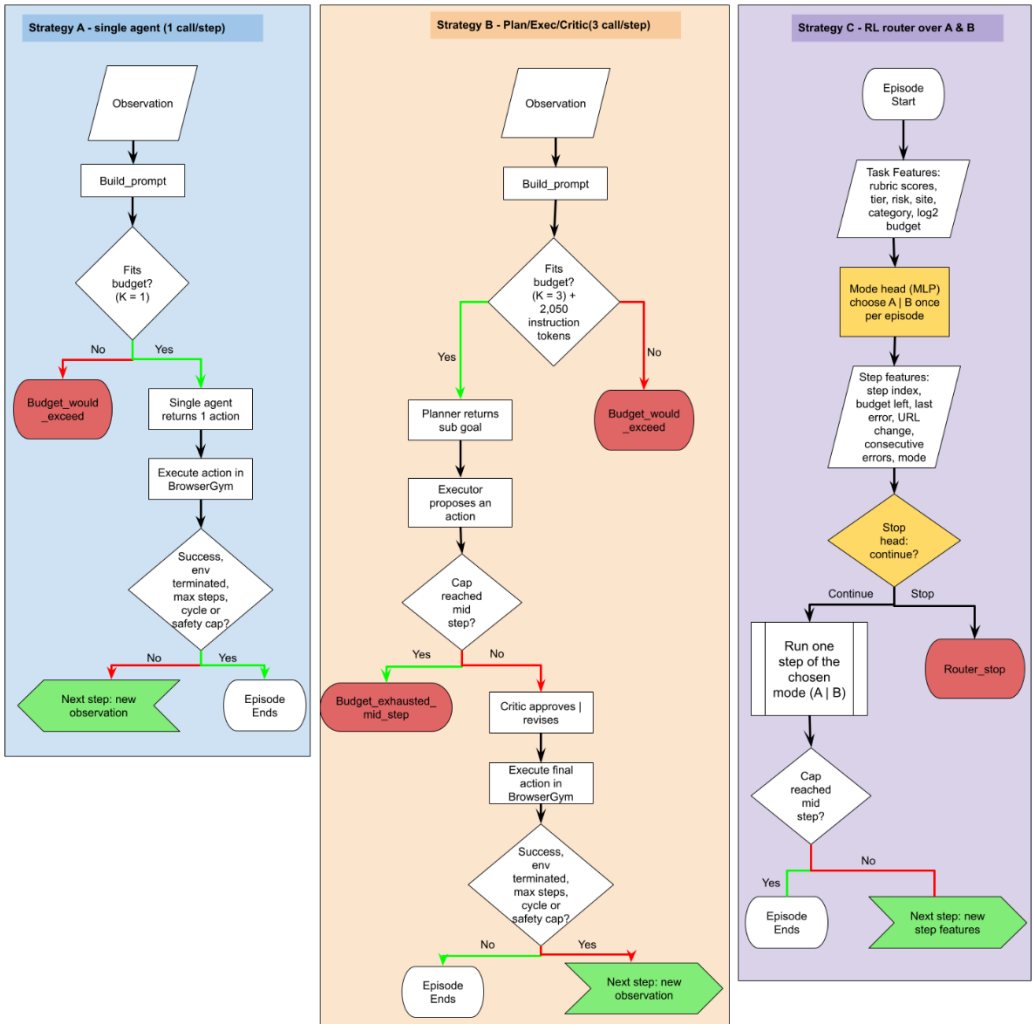


Figure 3.1 Per step control flow of the 3 strategies.

3.4.1 Strategy A – Single agent baseline

The single agent Strategy A is specified by a single action block of about 800 tokens (Listing B.5.2). That block, with examples and specification for answers, constitutes the vocabulary of actions; each step in the process of implementing that vocabulary invokes the agent once to produce a single line justification followed by a single line of action to be taken immediately. Thus, that single line of action is the cost floor against which are measured other strategies to implement this same process. Although the same amount of instructional overhead is incurred

only once, the cost to the single agent of access to the complete page at each step of the process increases with page size, not with task complexity.

3.4.2 Strategy B – Fixed Planner / Executor / Critic pipeline

For strategy B, the interaction is distributed among three roles: Planner, Executor, and Critic (see Listings B.5.3–B.5.5 for their input/output instructions). Importantly, the Planner solves the same problem as Agent A, but instead of outputting actions, it generates a sub-goal as a plain English sentence. The Executor is then given the prompt and the corresponding sub-goal and outputs a single action. The Critic is given the prompt, the sub-goal, and the action proposed by the Executor, and either approves the action or outputs a different action, which can be from the current page. All interactions are logged against the corresponding role, thus allowing us to analyse the Critic's revision rate as well as the number of tokens allocated by each of the roles in Chapter 4. Each of the three interactions re-sends the same page, hence their cost approximates that of 3A steps plus 2,050 instruction tokens. Budget verification with k = 3 also is restricted to verifying the cost of an entire cycle of interactions and thus will report budget_exhausted_mid_step if the budget runs out in the middle of a call.

The programming instructions for interacting with the environment went through 3 revisions. The 2nd version of the programming instructions for interacting with the environment was expanded but unfortunately did not include full interpolation of action vocabulary resulting in all 3 agents' generating hallucinatory action syntax. This data was only used as evidence and all subsequent uses of the programming instructions for interacting with the environment (i.e. the 3rd version which was significantly condensed and much easier to follow than the 2nd version) resulted in data that was used in analyses reported here.

3.4.3 Strategy C – Offline Reinforcement Learning dynamic system

Strategy C adds a router that makes two kinds of decision and otherwise runs the A or B machinery unchanged (Figure 3.2). The mode head decides once, at the start of an episode, whether to run in execute mode (A's single agent) or cycle mode (B's three agents); the stop head decides before every step whether to continue or to end the episode with router_stop, spending nothing further on it. Both heads are small multilayer perceptron (two hidden layers of 64 ReLU units) trained offline on the logged A and B episodes of the main grid, in the setting of offline reinforcement learning (Levine et al., 2020): no interaction with the sites was needed to train them.

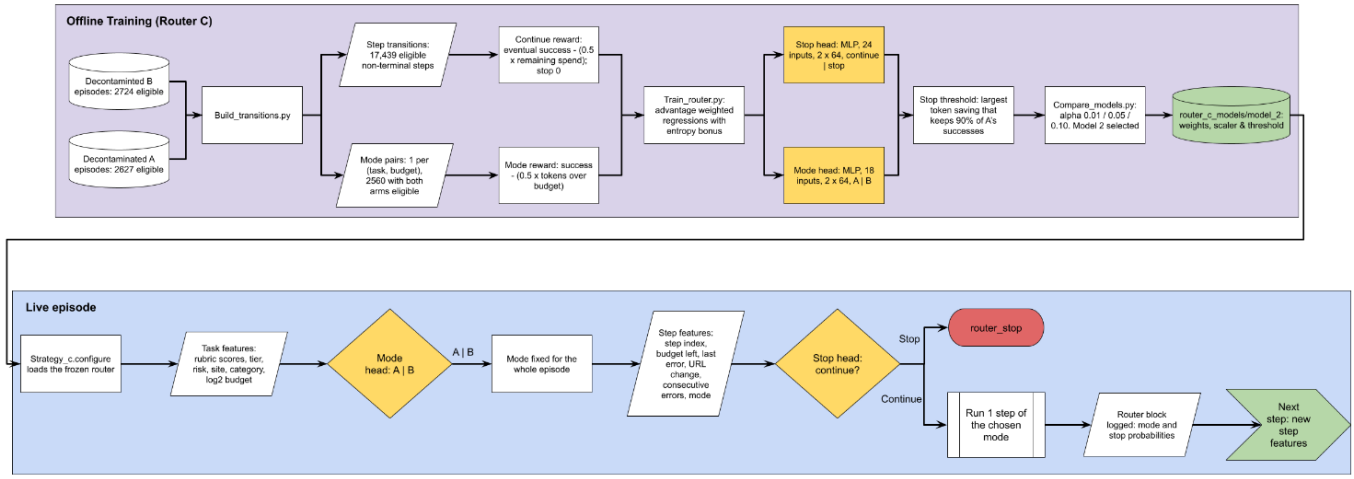


Figure 3.2: Strategy C: Training and their use during a live episode

Training data. The decontaminated A and B episodes were converted into two files: mode pairs, one row per (task, budget) with A's and B's success and tokens side by side (2,850 pairs, 2,560 eligible on both sides), and transitions, one row per site step of every episode plus a terminal row (23,801 rows, of which 17,439 non-terminal eligible rows train the stop head). Mode features describe the task and budget (rubric scores, tier, risk level, log2 budget, site and category: 18 inputs); stop features add the step index, budget remaining, last-action error, URL change, consecutive errors and mode (24 inputs). Inputs are standardised with statistics from the training folds only.

Rewards. Each head learns from a return that trades success against spend (equation 3.2):

$$R_{\text{mode}}(a) = s_a\left(1 - \lambda\frac{T_a}{B}\right), \qquad R_{\text{cont}} = s - \lambda\frac{T_{\text{rem}}}{B}, \qquad R_{\text{stop}} = 0 \qquad (3.2)$$

where s_a is arm a's success on the pair (0 or 1) and T_a its tokens, s and T_{rem} the eventual success of the logged episode and the tokens it went on to spend after the current step, B the budget and λ = 0.5. Stopping earns nothing, so continuing is rewarded only where the eventual success outweighs the discounted remaining spend.

Objective. Both heads are trained by advantage-weighted regression (Peng et al., 2019) with an entropy bonus (Equation 3.3):

$$J(\theta) = \mathbb{E}[\, w(s,a)\log\pi_\theta(a \mid s)\,] + \alpha\, H(\pi_\theta), \qquad w(s,a) = \min\left(e^{A(s,a)/\beta},\, 20\right) \qquad (3.3)$$

where A(s, a) is an action's return minus the mean return of the state's actions, β = 0.5 the advantage temperature, H the policy entropy and α the entropy weight; networks were trained with Adam (Kingma and Ba, 2015) at a learning rate of 0.001 for 60 epochs in batches of 256.

Evaluation and model selection. Tasks were dealt into five folds so that all six budgets of a task fall in the same fold and every prediction comes from a router that never saw the task. The held-out predictions were replayed against the logged episodes: the mode head's choice selects A's or B's episode for each (task, budget), and the stop head is applied to it step by step, charging the tokens spent up to a stop and counting no success. Sweeping the stop threshold traces a frontier of successes kept against tokens saved relative to always running A; the

threshold adopted is the largest saving that keeps at least 90% of A's successes (the test for Objective 3). Three routers differing only in α were trained between 24 and 27 September (Table 3.1); their offline results are nearly identical, so the choice is insensitive to α, and model 2 (α = 0.05), which retained the most successes, was refitted on all the data and deployed as Strategy C.

Table 3.1 The 3 routers trained, evaluated offline by 5-fold task level cross validation and replay against the logged episodes.

Model	α	Mode head AUC	Stop head AUC	Successes kept	Tokens saved
1	0.01	0.810	0.753	91.2%	22.4%
2	0.05	0.811	0.756	91.8%	22.0%
3	0.10	0.812	0.757	91.0%	22.1%

3.5 Data collection, logging and task classification

Collections were executed by running run_batch.py on an appropriate instance. The methodology behind the script run (a) for each (task, budget) cell not yet written to output file, group cells by task. Budgets within a task as well as order of tasks were all randomized using a fixed seed (42). Each episode generated one JSON record, which could then be processed. Budgets within a task were randomized to ensure the first run for a task did not systematically map to the cheapest option. However, as completed cells are not written when an instance crashes or API call fails, work not already performed would be skipped on restart. Errors due to environmental issues (e.g. page load timeouts, Chromium crashes) were removed using remove_errors.py, and the same command re-run to re-collect the necessary information. (As sites were re-created from the base images of the instances from which they ran, no runs would have encountered effects of previous runs at the instance level.) All these runs are summarized in Table 3.2.

Table 3.2 The collection runs. Eligible episodes are those retained for results after decontamination

Run	Episodes	Tokens	Eligible
Strategy A, main grid (29 July – 12 Aug)	2,850	29.5 M	2,627 (92.2%)
Strategy B, main grid (7 – 12 Aug)	2,850	44.3 M	2,723 (95.5%)
Strategy C, main grid (25 – 28 Sep)	2,850	23.7 M	2,604 (91.4%)
Spending Schemes (28 – 29 Sep)	900	18.4 M	900 (read only)

3.5.1 Task classification

Two labels were attached to every task before the episodes were cleaned. A task category (information retrieval, navigation, create, modify value, delete, bulk action, purchase, other) was assigned by keyword rules over the task's intent, checking read-only phrasing first so that a question mentioning "post" or "purchase" is not filed as a write. A contamination-risk level was then assigned: read-only tasks change nothing; idempotent tasks (wishlist, cart, subscribe, vote) converge to the same state however often they are repeated; non-idempotent tasks

(orders, posts, edits, deletions) leave the site changed. The 27 tasks whose evaluator accepts only "N/A" cannot change state and were forced to read-only. The rules were tested against 29 hand-checked intents (Section 3.6). Table 3.3 gives the result: just over half of the tasks cannot be contaminated at all, but the forum's tasks are mostly writes.

Table 3.3 Contamination risk level by site

Site	Read-only	Idempotent	Non-idempotent	Total
Shopping	131	21	35	187
Shopping admin	101	0	81	182
Forum	11	15	80	106
Total	243 (51%)	36 (8%)	196 (41%)	475

3.5.2 Decontamination

The containers for WebArena preserve the state of episodes between runs. Each task is then executed 6 times for each strategy on a shared site instance. For writing tasks, subsequent attempts after a successful execution of a task will find the work has already been done. The attempt may then result in trivial success or failure. In both cases, it is unclear whether the agent can solve the task from scratch. A pipeline of 5 scripts is used to handle these issues (see Figure 3.3). The scripts are run in sequence: remove_errors (Section 3.5), sort_batch (reverses the tasks to the order in which they were placed in the batch for resumed execution of episodes), classify_tasks (Section 3.5.1), flag_episodes, and join_metadata.

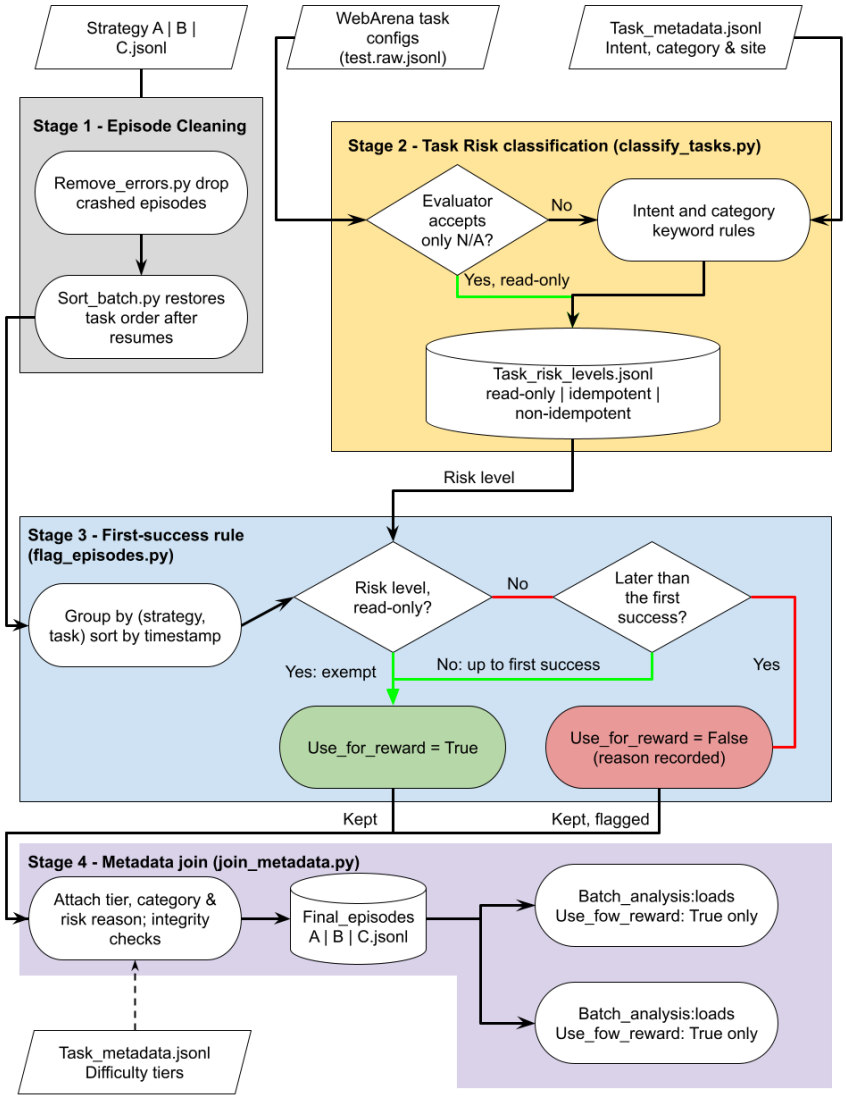


Figure 3.3 Decontamination pipeline

flag_episodes.py applies one rule. Episodes are grouped by strategy and task and ordered by timestamp; for idempotent and non-idempotent tasks every episode after the first success is marked ineligible for reward, with the reason recorded, while read-only tasks are exempt. Grouping by strategy means one strategy's success never disqualifies another's episodes, which the site rebuilds between collections make correct. Flagged episodes are kept in the files rather than deleted; the analysis loader drops them, and the exclusions amounted to between 4.5% and 8.6% of each strategy's episodes (Table 3.5). join_metadata.py then attaches the category, risk reason and difficulty fields and checks that every episode's site and goal agree with its task configuration.

3.5.3 Difficulty annotation

RQ2 needs a difficulty label that does not depend on how the agents fared. Each task was scored on a four-dimension rubric (Table 3.4), each dimension from 0 to 2, and the total of 0 to 8 banded into Easy (0 to 2), Medium (3 to 5) and Hard (6 to 8). Scoring 475 tasks by hand was not feasible, so an LLM annotator (GPT-5 mini, through the same typed-output mechanism as the

agents) scored every task from its intent and evaluation criteria, with Strategy A's observed cost on the task (median and maximum steps, distinct pages visited) as a calibration hint for the pages dimension only (Listing B.5.6). Tasks sharing an intent template were harmonised to the template's median scores.

Table 3.4 The difficulty rubric. Each dimension is scored 0-2; the total (0-8) is banded into Easy (0-2), Medium (3-5) and Hard (6-8)

Dimension	0	1	2
Pages to traverse	One page	Two or Three pages	Four or more, or unbounded
Retrieval type	Read one value	Compare or filter a few items	Aggregate or count over a set
Interaction	Read-only	One form or click sequence	Multi-step state change
Target locatability	Named in the task	Derivable from the page	Must be discovered by scanning

The annotator was validated against hand labels between 18 and 24 September. I labelled a site-stratified gold set of 50 tasks; 20 were placed in the annotator's prompt as calibration examples and 30 were held out and never shown to it. Agreement is measured with Cohen's kappa (Cohen, 1960), chance-corrected agreement (Equation 3.4), in its quadratic-weighted form (Cohen, 1968) because the scores are ordinal:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \qquad (3.4)$$

where p_o is the observed agreement and p_e the agreement expected from the raters' marginal distributions. On the 30 held-out tasks the mean quadratic kappa across the four dimensions was 0.63, "substantial" on the Landis and Koch (1977) scale, against 0.87 on the 20 exemplars (Table 3.5); the held-out figure is the one reported, since the exemplar figure is inflated by exposure, and pages to traverse was the weakest dimension, as expected for the quantity hardest to judge from text. The 50 hand labels then replaced the model's scores for those tasks, with the provenance recorded per task, giving 156 Easy, 250 Medium and 69 Hard tasks (33%, 53% and 15%), spread similarly across the three sites (Figure 3.4).

Table 3.5 Agreement between hand labels and the LLM annotator (quadratic-weighted kappa per dimension)

Measure	**Held out (n = 30)**	**Exemplars (n = 20)**	**All (n = 50)**
Pages to traves	0.43	0.79	0.64
Retrieval type	0.60	0.88	0.77
Interaction	0.74	0.85	0.80
Target Locatability	0.76	0.93	0.85
Mean of four dimensions	0.63	0.87	0.76
Tier agreement (exact)	67%	90%	76%

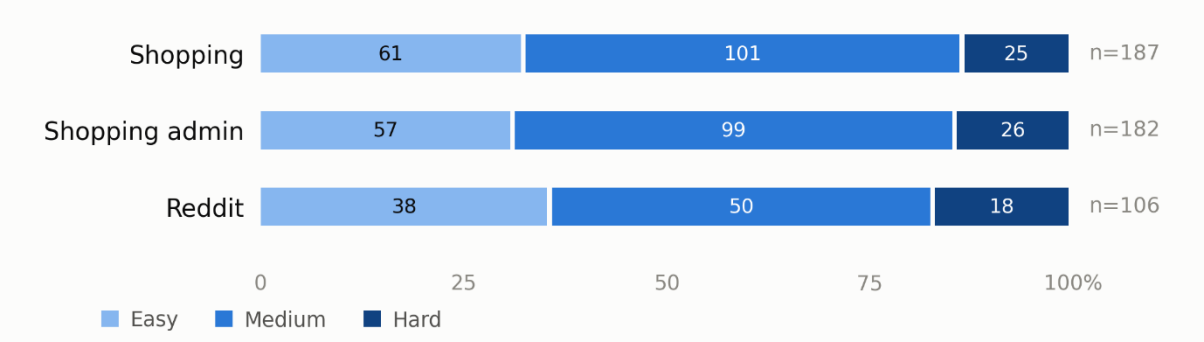


Figure 3.4 Difficulty tier composition by site after the 50 gold labels were applied

3.6 Verification and validation

Verification was built into each layer of the system. Harness smoke tests for one task per site were run prior to data collection for each task. Potential stock behaviours which could cause results to be invalid for a given task were identified and fixed for the following behaviours: 500ms action timeout treating successful clicks as failed clicks, the deprecated evaluator model, and the evaluator which could not handle dictionary answers. A runtime guard to prevent Strategy B from being run if any of the instruction blocks for a given task did not have the proper action vocabulary for that task was added for the 2nd version of instructions for that task.

The rule-based classifiers have self-testing in place for classify_tasks.py (29 hand-verified intents, including homographs i.e. 'remover'/'remove' and quoted payloads) and flag_episodes.py (first-success on synthetic episodes).

The gold set was verified for internal consistency before use. join_metadata.py verifies that each episode was for the correct task, based on the corresponding task configuration for that episode.

Budget verification for collected data verifies that no episode went over its cap by more than one step's output. The router for live mode was validated via cross-validation and then offline-replay prior to collecting any live episodes for this task (Section 3.4.3), and the choices made and the stopping steps for cases where offline-prediction was used were logged for later comparison with the results of the same offline code.

Statistical treatment. Success rates are reported with Wilson (1927) 95% intervals (Equation 3.5), which stay inside [0, 1] and behave correctly at the near-zero rates of the 2k and 4k budgets:

$$\tilde{p}_{\pm} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \qquad \hat{p} = \frac{k}{n},\ z = 1.96 \qquad (3.5)$$

Strategies are compared on the same (task, budget) cells with the exact McNemar test (McNemar, 1947), which uses only the discordant cells, the b solved by one strategy alone and the c solved by the other alone (Equation 3.6):

$$p = \min\left(1,\ 2\sum_{i=0}^{\min(b,c)}\binom{b+c}{i}(1/2)^{b+c}\right) \qquad (3.6)$$

Because several pairs and tiers are tested, thresholds are Bonferroni-corrected (Dunn, 1961): 0.05/3 for the three pooled pairs and 0.05/9 within tiers. Paired, skewed token differences between schemes use the Wilcoxon signed-rank test (Wilcoxon, 1945); the association between failure mode and tier uses Pearson's chi-square test (Pearson, 1900); and difficulty sensitivity and scheme effects carry intervals from a task-level bootstrap of 2,000 resamples (Efron, 1979). Tests were computed with SciPy (Virtanen et al., 2020), apart from the Wilson interval and exact McNemar, which were implemented directly.

3.7 Ethics, legal, social and professional issues

There were no human participants in the project and no personal data used – the Research Ethics Checklist (Appendix E.1) identified no participants and minimal risk. All of the agent actions were performed on self-hosted replicas of the four sites running with data provided by WebArena. The sites were recreated from scratch between each run. The API key and the instance key pair for the tests were kept outside of version control, access to the instance via SSH was restricted to a single IP address and the repository for the project is publicly available on the web. WebArena and BrowserGym are distributed under open-source licenses which allows for their use in research. The task configurations used in this project are attributed to WebArena and BrowserGym as specified in Appendix C.

Budget for the cost of collections was approached in a professional manner: although the instance was switched off between runs where possible, failed instances of episodes were re-collected instead of re-running the whole task. Generative AI was used according to the school's policy, with prompts and their validation described in detail in Chapter 5 and Appendix D. Note also that the findings have been restricted by several severe limitations: a single big budgetary error, severe constraints on the three sites and one model scope, and a heavily shopping biased sample for Research Question 3. All of these are detailed here and re-described in detail in Chapter 6.

4 Results

In this chapter, results from the experiments are reported. All numbers stem from the analysis scripts provided in the repository, i.e., they were computed on the decontaminated episode files as described in Section 3.5.2. Printed outputs of the analysis scripts as well as resulting figures were collected in Appendix A. All success rates are supplemented by corresponding Wilson intervals for 95% coverage. All differences between strategies were tested using exact McNemar tests, which are paired for corresponding (task, budget) cells that were attempted by both strategies. The tests are evaluated against corresponding Bonferroni bounds, as specified in Section 3.6: 0.0167 for the three pooled tests, 0.0056 for the nine tier tests, 0.0028 for the 18 per-budget tests as well as for the 18 within-strategy scheme tests, and 0.0042 for the 12 strategy pairs for each of the schemes.

4.1 What was produced

The five tangible outputs presented in this report were all produced by the author, while WebArena and BrowserGym only provided the environment for the work to take place and the respective tasks that were completed.

1. The harness and the three strategies (wa_env.py, strategy_a.py, strategy_b.py, strategy_c.py, run_batch.py).

2. The episode dataset. This dataset contains 8,550 main-grid episodes (for 475 tasks, 6 budgets, 3 strategies) and 900 episodes for the spending-scheme, summing up to approximately 116 million logged tokens (see Table 3.6). Each row of the resulting table contains the goal, a per-call step log, the termination reason, the contamination verdict, and several metadata items regarding difficulty (a schema for this data is described in Appendix C.1). After decontamination, 7,954 main-grid episodes remain for analysis.

3. Metadata for all 475 tasks (categories, risk scores, rubric scores, tiers and their sources).

4. The trained router (model 2) including scaler and stop threshold values as well as training data set (2,560 mode pairs / 17,439 step transitions).

5. Scripts to perform the analysis, output from said scripts, and an online repository containing this information, and a means to reproduce each of the modules individually (Appendix B).

4.2 The collected dataset

Table 4.1 summarises the eligible data. Strategy B lost the fewest episodes to the first-success rule because it succeeded least on state-changing tasks; A and C lost 223 and 246. Two properties shape everything that follows. First, a Strategy B step costs 2.8 times a Strategy A step (median 6,967 against 2,480 tokens), the arithmetic of three calls on the same page plus 2,050 instruction tokens. Second, Strategy C's per-step cost is indistinguishable from A's, because its mode head chose A's single-agent mode in 2,599 of 2,604 episodes (Section 4.3.3). Per-site figures are in Appendix A.

Table 4.1 The eligible dataset by strategy. Success rate pooled over all six budgets with Wilson 95% intervals

Strategy	Episodes Kept	Successes	Success rate [CI]	Tasks solved at any budget	Median tokens per step	Tokens per success
A Single agent	2,627 (223)	383	14.6% [13.3, 16.0]	171 / 475 (36%)	2,480	73,490
B fixed pipeline	2,723 (127)	226	8.3% [7.3, 9.4]	120 / 475 (25%)	6,967	193,544
C routed	2,604 (246)	360	13.8% [12.6, 15.2]	157 / 475 (33%)	2,490	63,393

4.3 RQ1 – Success and cost against token budget

Table 4.2 and Figure 4.1 give the cost-performance curves that RQ1 asks for: success as a function of the fixed budget. Figure 4.2 replots the same successes against the tokens spent, which is where the strategies separate most clearly: A and C climb the same curve, and B reaches each level of success only at two to four times the spend.

Table 4.2 Success rate by budget with Wilson 95% intervals

Budget	A success [CI]	B success [CI]	C success [CI]	A median tokens	B median tokens	C median tokens
2k	1.4% [0.6, 2.9]	0.0% [0.0, 0.9]	1.4% [0.6, 3.0]	2,382	0	0
4k	5.5% [3.7, 8.0]	0.0% [0.0, 0.8]	6.1% [4.2, 8.8]	4,494	0	2,052
8k	12.5% [9.7, 15.9]	5.7% [3.9, 8.2]	11.0% [8.4, 14.3]	7,747	6,050	7,434
16k	21.1% [17.6, 25.1]	7.8% [5.7, 10.6]	17.3% [14.1, 21.1]	15,157	13,691	12,424
32k	23.5% [19.8, 27.7]	13.2% [10.4, 16.6]	23.5% [19.8, 27.7]	17,676	29,211	14,920
64k	23.6% [19.8, 27.9]	22.6% [19.0, 26.6]	22.9% [19.2, 27.1]	17,391	58,527	15,518

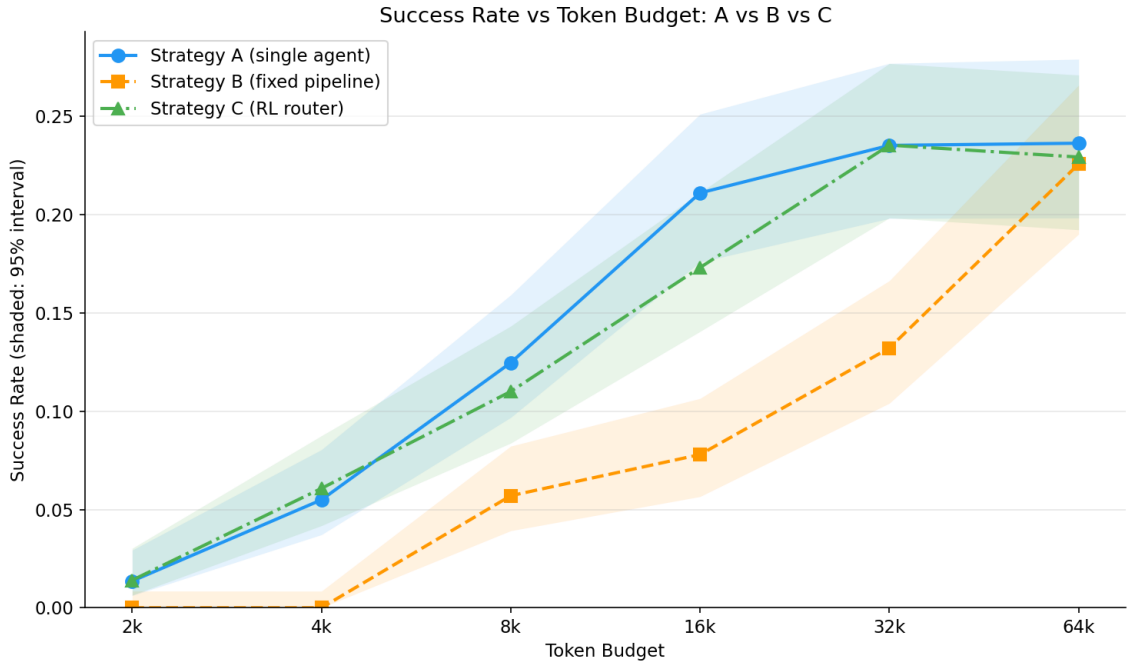


Figure 4.1 Success rate against token budget for the three strategies, with Wilson 95% bands

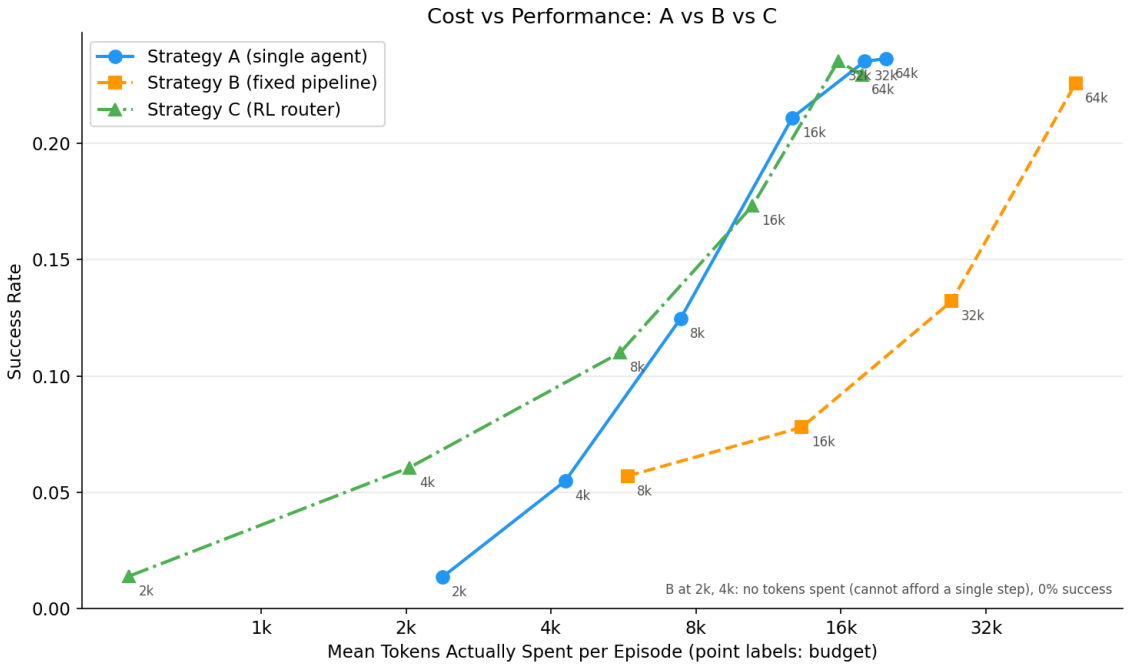


Figure 4.2 Success rate against the mean tokens spent per episode

4.3.1 Strategy A

The single agent's curve rises steeply to 16k and then flattens (21.1%, 23.5%, 23.6%), with overlapping intervals above 16k. The share of failures that were budget-driven falls from 100% at 2k and 98% at 4k to 66% at 16k, 21% at 32k and 2% at 64k, and mean utilisation of the 64k budget was 27%. What remains at large budgets is wrong answers and guard-fired stops: 422 episodes ended with an incorrect answer, after a median of 9,144 tokens and four steps, and 64% of them occurred at 32k or 64k. Successful episodes were cheap: median 6,570 tokens over two steps. A solved 171 of the 475 tasks at some budget; the median budget at which a task was first solved was 16k.

4.3.2 Strategy B

Strategy B solved nothing at 2k or 4k, not because it failed but because it never started: the pre-step check of Equation 3.1 (three prompts, 2,050 instruction tokens and the output margin) exceeded both budgets on every task, so 899 episodes ended with budget_would_exceed and zero spend. From 8k its success rate rose with budget, 5.7%, 7.8%, 13.2% and 22.6%, without the plateau A shows, because the budget was still binding at 64k: 86% of all B episodes ended for budget reasons, against 56% for A. Equivalent-budget matching (Appendix A.4) puts the overhead in one line: B at 64k matched A at 32k, B at 32k matched A at 8k, and B at 16k matched A at 4k, so B needed two to four times the budget for the same success. A successful B episode cost a median of 11,994 tokens, 1.8 times A's, and B solved 120 tasks at some budget.

4.3.3 Strategy C

Strategy C matched Strategy A within the confidence interval at every budget (Table 4.2) while spending less: its median and mean episode cost were below A's at every budget (median 14,920 against 17,676 at 32k). Table 4.3 makes the comparison on the 2,463 (task, budget) cells that all three strategies attempted. The live router kept 93% of A's successes (329 of 352) for 20% fewer tokens, between the always-A and always-B policies and far from the hindsight oracle, which marks the headroom a perfect stop decision would leave. The offline replay of Section 3.4.3 had predicted 91.8% and 22.0% (Table 3.5), so the frozen router transferred to live episodes within two points on both axes, and the Objective 3 test of at least 90% was passed live as well as offline.

Table 4.3 Strategy C against fixed policies on the 2,463 cells shared by A, B & C

Policy	Successes	Kept vs always A	Tokens	Saved vs Always A
Always A	352	100%	26,582,476	0%
Always B	190	54%	39,448,582	-48%
Strategy C (live router)	329	93%	21,357,941	20%
Oracle (Hindsight)	372	106%	3,444,755	87%

The saving came almost entirely from the stop head. It ended 838 episodes (32%), 703 of them before any action was taken, and its stop rate fell with budget from 77% at 2k to 55% at 4k, 27% at 8k and 8.5% at 64k (Appendix A.3): at small budgets it declined most tasks, at large budgets it let the agent run. Among the episodes it allowed to proceed, success was higher than A's at five of the six budgets (Figure 4.3), 26.5% against 23.5% at 32k and 20.6% against 21.1% at 16k, which is what a stop decision that removes mostly hopeless episodes should produce. The mode head, by contrast, chose B's pipeline in only five of 2,604 episodes, all on one shopping-admin template family (tasks 459 to 463) that B solves and A fails by exhausting its 25 steps, and three of the five succeeded. In practice, therefore, Strategy C is the single agent plus a learned decision about whether to attempt a task; the data it was trained on (Section 4.3.4) left the mode head little else to learn.

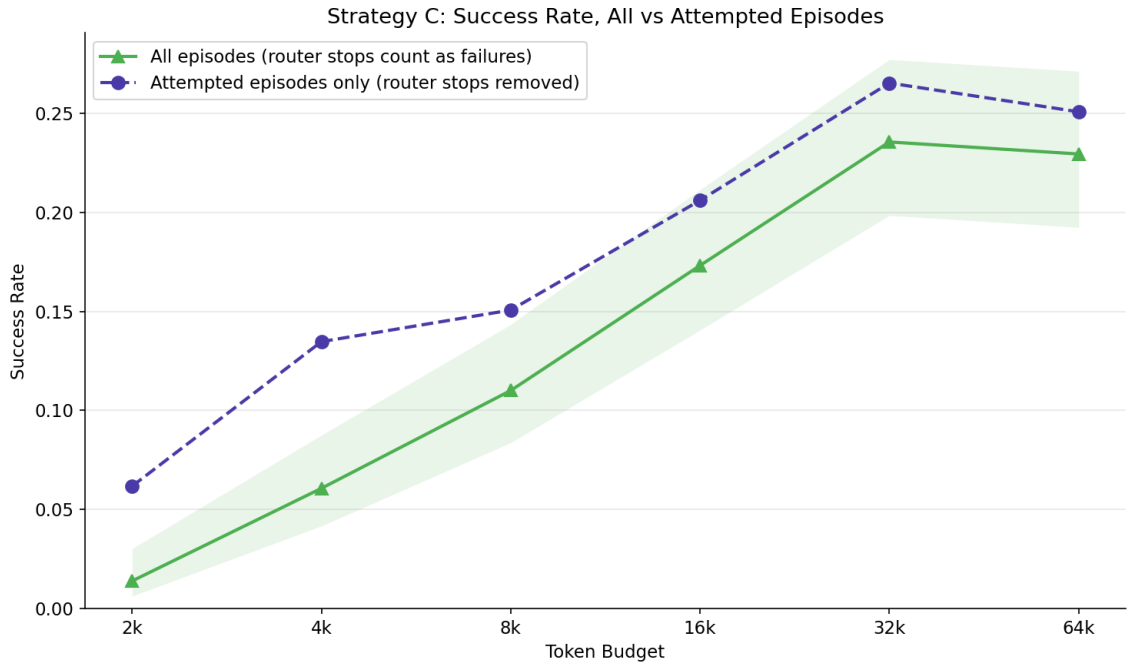


Figure 4.3 Strategy C success rate counting router stops as failure (solid) and on attempted episodes only (dashed)

4.3.4 Matched budget comparison

Table 4.4 gives the paired tests. Pooled over the grid, A beat B (192 cells A alone against 24 B alone, $p < 0.0001$) and C beat B (183 against 34, $p < 0.0001$), while A and C did not differ (83 against 60, $p = 0.065$). Per budget, A's advantage over B and C's advantage over B were significant from 4k to 32k at the 0.0028 threshold; at 2k the discordant counts were too few (6 and 6) and at 64k the gap had closed ($p = 0.033$ and 0.229). A and C were not distinguishable at any budget (smallest $p = 0.13$). On tasks that both strategies solved in the same cell, B spent a median of 2.90 times A's tokens (105 tasks), whereas C spent 1.00 times A's (135 tasks). At the task level, 95 tasks were solved by all three strategies at some budget, 272 (57%) by none, 26 by A alone, 17 by C alone and 10 by B alone (Appendix A.4).

Table 4.4 Exact McNemar tests paired on (task, budget) cells. 'A only' counts cells A solved and B did not. Threshold 0.0167 for the pooled rows, 0.0028 per budget.

Budget	A vs B: A only / B only	p	A vs C: A only / C only	p	B vs C: B only / C only	p
2k	6 / 0	0.031	4 / 4	1.000	0 / 6	0.031
4k	24 / 0	< 0.0001	0 / 1	1.000	0 / 25	< 0.0001
8k	30 / 0	< 0.0001	9 / 3	0.146	2 / 25	< 0.0001
16k	52 / 0	< 0.0001	23 / 13	0.133	3 / 42	< 0.0001
32k	47 / 7	< 0.0001	23 / 24	1.000	6 / 52	< 0.0001
64k	33 / 17	0.033	24 / 15	0.200	23 / 33	0.229
pooled	192 / 24	< 0.0001	83 / 60	0.065	34 / 183	< 0.0001

4.3.5 Failure modes

Figure 4.4 shows how episodes ended. Two-thirds of A's 2,244 failures were budget-driven (66%), 19% were wrong answers and 15% guard stops (navigation cycles and repeated failures). B's 2,497 failures were almost all budget-driven (94%); it rarely reached the point of answering wrongly (3%) or looping (2%). C's failures split four ways: 31% budget, 37% router stop, 18% wrong answer and 13% guards, so the router replaced roughly half of A's budget exhaustion with a cheaper, earlier exit. Token efficiency, successes per thousand tokens, peaked at 8k for A (0.017) and B (0.010) and at 4k for C (0.030), and fell at larger budgets for all three (Appendix A.4).

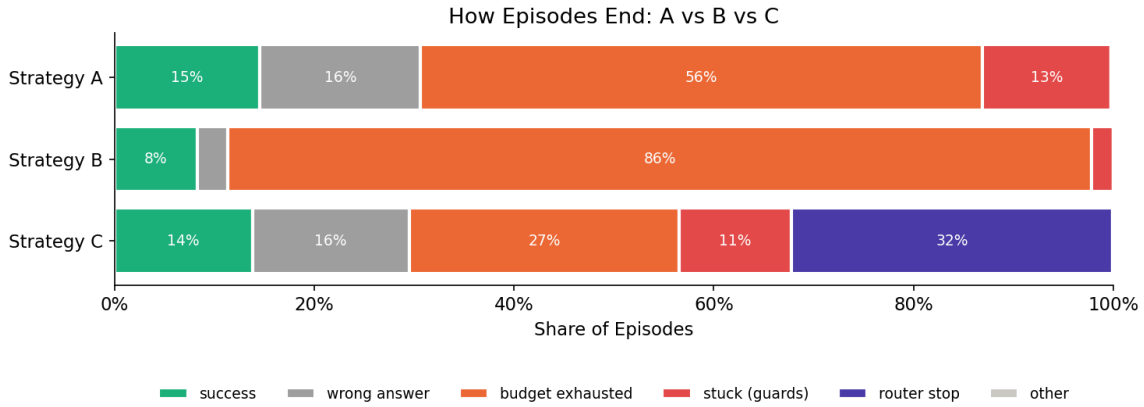


Figure 4.4 How episodes ended, as a share of all eligible episodes per strategy

4.4 RQ2 – Sensitivity to task difficulty

Table 4.5 answers RQ2 at the tier level. Every strategy's success fell from Easy to Hard, and the ordering of the strategies was the same in every tier: A beat B in all three ($p < 0.0001$ each, against 0.0056 for nine tier tests), C beat B in all three ($p < 0.0001$), and A and C did not differ ($p = 0.058$, 0.434 and 1.000). Sensitivity, measured as Hard-tier success as a share of Easy-tier success with task-level bootstrap intervals, was 0.55 for A, 0.42 for B and 0.62 for C; the paired differences were B minus A -0.13 [-0.35, +0.08] and C minus A +0.07 [-0.03, +0.23], neither clear, and C minus B +0.20 [+0.01, +0.44], which excludes zero. The pipeline, then, was the most difficulty-sensitive strategy and the router the least, but only the C against B contrast is supported. By budget within tier (Appendix A.4), B overtook A on Easy tasks only at 64k (31.3% against 30.1%) and never on Medium or Hard tasks.

Table 4.5 Success rate by difficulty tier pooled over budgets, with Wilson 95% intervals and the paired McNemar p-value within each tier (threshold 0.0056).

Tier	A [CI]	B [CI]	C [CI]	A vs B p	A vs C p	B vs C p
Easy (n = 805 / 842 / 793)	18.3% [15.7, 21.1]	10.9% [9.0, 13.2]	16.5% [14.1, 19.3]	< 0.0001	0.058	< 0.0001
Medium (1,422 / 1,467 / 1,412)	13.8% [12.1, 15.7]	7.8% [6.6, 9.3]	13.3% [11.6, 15.2]	< 0.0001	0.434	< 0.0001
Hard (400 / 414 / 399)	10.0% [7.4, 13.3]	4.6% [3.0, 7.1]	10.3% [7.7, 13.6]	< 0.0001	1.000	< 0.0001

Hard / Easy ratio [bootstrap CI]	0.55 [0.25, 0.94]	0.42 [0.13, 0.86]	0.62 [0.28, 1.09]	-	-	-

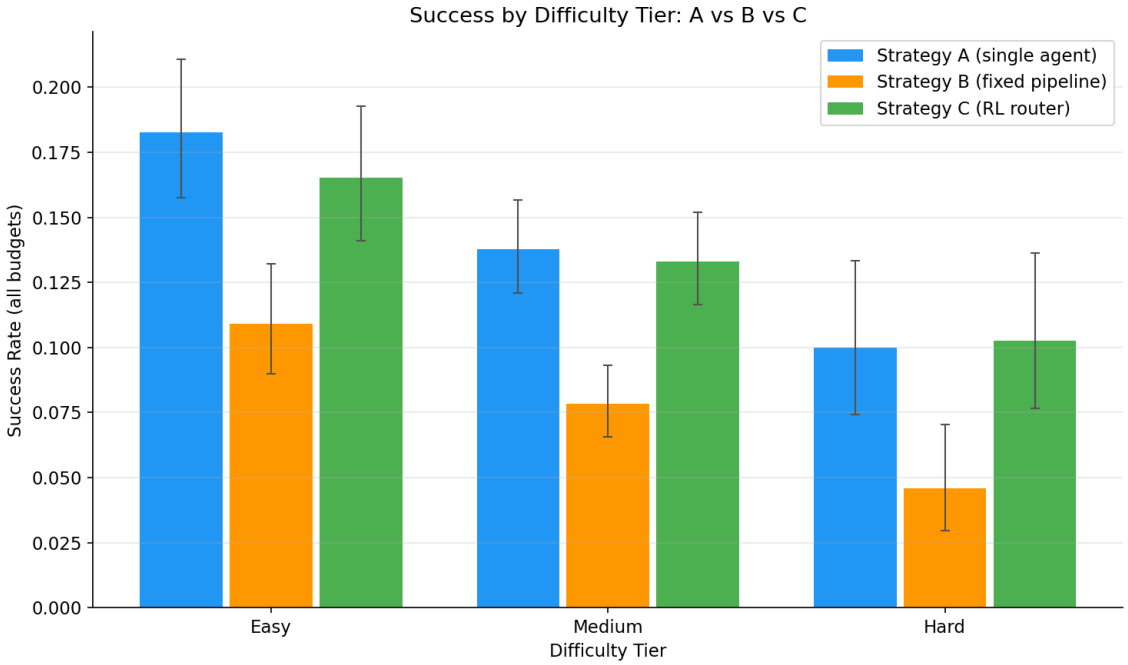


Figure 4.5 Success rate by difficulty tier with Wilson 95% intervals

4.4.1 Tasks solved by the pipeline and not by the baseline

Fifteen tasks were solved by the pipeline at some budget and never by the single agent (Appendix A.4 lists them). All fifteen were solved at 32k or 64k in three to ten steps; eleven succeeded with the Planner's sub-goals alone and four involved a Critic revision. A's failures on the same tasks were not budget exhaustion but loops and crashes: navigation_cycle on six, env_terminated on four and max_steps on five, the last five being the shopping-admin template family that C's mode head also routed to the pipeline. Decomposition therefore helped where the single agent kept returning to the same page, not where it ran out of tokens. The reverse set is larger: A solved 36 tasks that C never did; C's episodes on them ended by budget exhaustion (72), router stop (63), wrong answer (43) and guards (37), so part of the stop head's saving was paid in declined tasks.

4.4.2 Characteristic failures

The failure mix changed with difficulty for every strategy (Pearson chi-square: A $\chi^2 = 19.8$, $p = 0.003$; B $\chi^2 = 9.5$, $p = 0.050$; C $\chi^2 = 72.6$, $p < 0.0001$; Appendix A.4). For A the change was a rising share of wrong answers, 15% of Easy failures to 23% of Hard ones, with budget exhaustion steady at 63 to 69%. For B the mix barely moved: 93 to 96% budget exhaustion in every tier. For C the change was the router: stops made up 32% of Easy failures, 36% of Medium and 52% of Hard, and its stop rate on Hard tasks (46%) was almost twice that on Easy tasks (27%): the stop head learned to decline the tasks on which A's success was lowest.

4.5 Critic behaviour in Strategy B

The Critic was called 6,263 times and revised the Executor's action 322 times, a revision rate of 5.1%; 94.9% of its calls approved the action unchanged. Revisions were concentrated at large budgets and late steps: 1.0% of calls at 8k, 1.7% at 16k, 4.0% at 32k and 7.3% at 64k (Figure 4.6), at a median step of four with 31% of the budget remaining. The revised actions were usually executable, 210 of 322 (65%) ran without an action error, but they rarely rescued the episode: only 16 revisions (5%) occurred in episodes that went on to succeed, episodes containing a revision succeeded less often than those without (4.8% against 8.6%), and 215 of B's 226 successes (95%) involved no revision at all. One signal predicted a useful call: 6% of revisions followed an action error, against a 2% base rate of post-error calls (lift 2.6). The cost of this review was 13.76 million tokens, 31.5% of everything B spent, of which 13.00 million (29.7% of B's total) went on approvals that changed nothing. Per logical step the three roles cost 2,661 (Planner), 2,125 (Executor) and 2,198 (Critic) tokens, 6,984 in all.

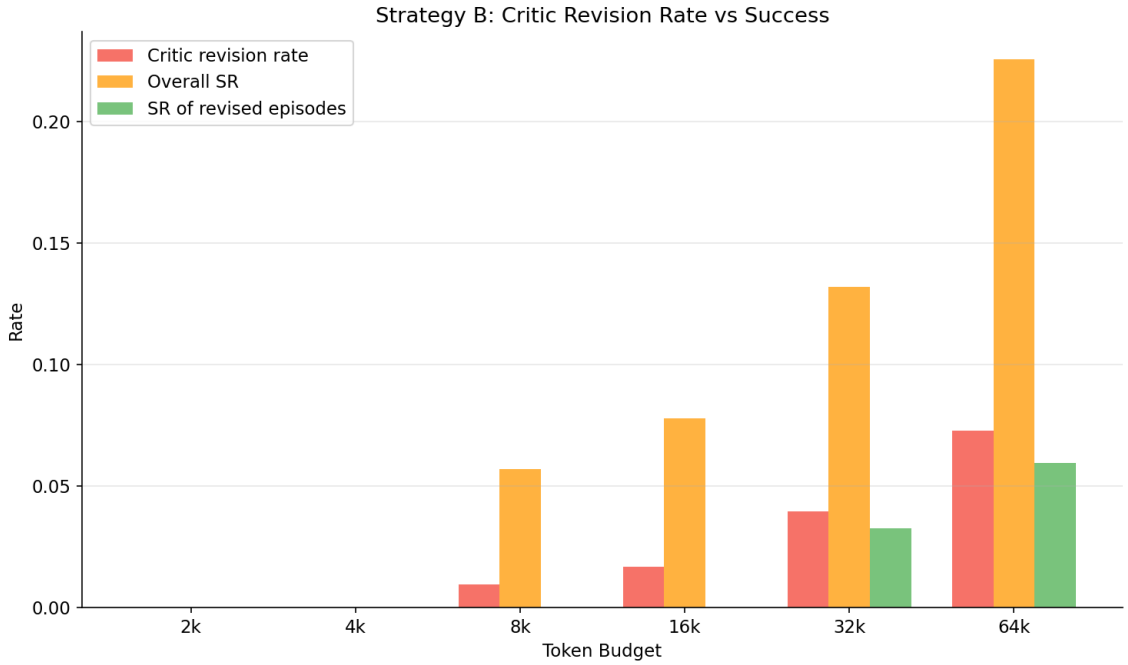


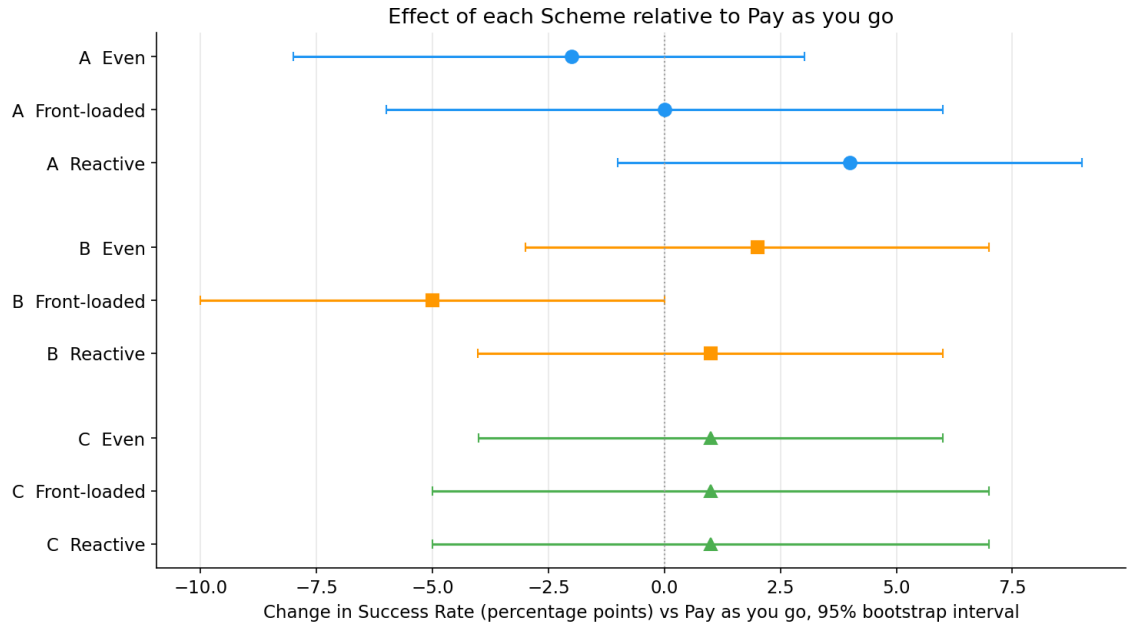
Figure 4.6 Strategy B's Critic revision rate, overall success rate and the success rate of episodes containing a revision, by budget

4.6 RQ3 – Intra-episode allocation

The RQ3 grid for the 100 read-only tasks at 32k is given in full in Table A.5.1 and the bootstrap effects in Table A.5.2; pay-as-you-go is the main-grid reference. Success ranged from 20 to 26% for A, 7 to 14% for B and 22 to 23% for C, at mean spends of 15.4k to 17.4k, 27.6k to 29.6k and 14.6k to 17.3k tokens per episode. No scheme changed any strategy's success rate: none of the 18 within-strategy McNemar tests reached the 0.0028 threshold (smallest p = 0.031, B reactive against front-loaded), and every bootstrap interval for the change in success spans zero (Table A.5.2, Figure 4.7). What the schemes changed was spend. For B, all three schemes cost more than pay-as-you-go with intervals excluding zero, even +6.2% [+2.9, +9.8], front-loaded +3.3% [+0.6, +6.3] and reactive +7.3% [+5.2, +9.5], because fitting a three-call cycle into a per-step allowance trimmed the page from 49 to 78% of steps and the pipeline then took more steps. For

A, front-loaded and reactive raised spend through their high-effort steps (+9.6% and +11.9%, both intervals reaching zero; Wilcoxon against the even scheme p = 0.0004 and 0.0017). For C, even and front-loaded trended cheaper (-6.3% and -7.0%) with intervals that include zero.

The ordering between strategies survived every scheme: at the 0.0042 threshold A beat B under pay-as-you-go and front-loaded (p = 0.002 and 0.0003) and C beat B under front-loaded (p = 0.0001), the other B contrasts did not reach it (p = 0.007 to 0.18), and A and C never differed (p >= 0.375; Appendix A.5). Descriptively, the reactive scheme gave A its best cell (26%, 67,066 tokens per success) at the price of more budget-exhausted episodes (21% against 9%), the even scheme suited B (14%), and the front-loaded scheme was C's cheapest per success (63,471 tokens) while it was B's worst (7%), since spending 40% of the budget on the first cycle left too little for the rest.



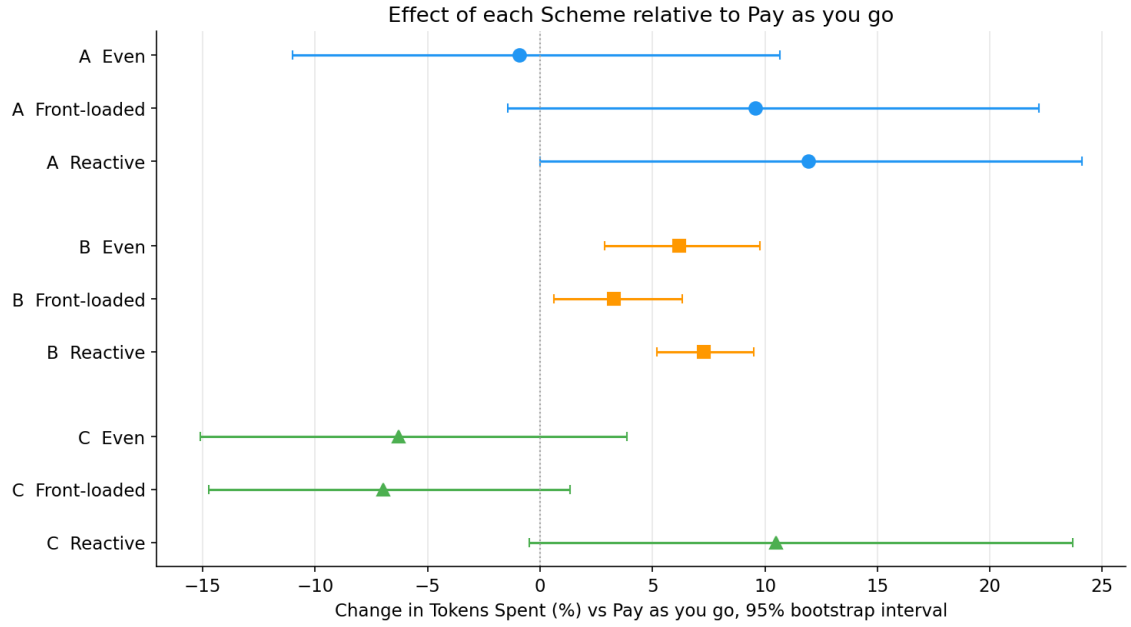


Figure 4.7: Change in success rate (top) and in tokens spent (bottom) for each scheme relative to pay-as-you-go, with 95% bootstrap intervals

4.7 Summary of results

RQ1. Success rises steeply to 16k and then saturates near 23.5% for the single agent and the router. The fixed pipeline cannot act below 8k and needs two to four times the budget to match the single agent, because each of its step's costs 2.8x as much and its Critic approves 95% of what it sees. The router kept 93% of the single agent's successes for 20% fewer tokens, within two points of its offline prediction, and was indistinguishable from the single agent at every budget.

RQ2. All strategies lose success from Easy to Hard tasks. The pipeline is the most sensitive (Hard/Easy 0.42) and the router the least (0.62), the only supported contrast being router against pipeline. The pipeline's 15 unique solutions are tasks on which the single agent loops rather than runs out of budget; the router's savings are concentrated on hard tasks, which it declines twice as often as easy ones.

RQ3. No allocation scheme changed any strategy's success rate. Schemes changed spend: every scheme made the pipeline dearer, the high-effort schemes made the single agent dearer, and the even and front-loaded schemes made the router slightly cheaper. The strategy ordering was the same under all four schemes.

Against the tests of Section 1.3: every strategy ran end to end with the common schema (Objective 1); the dataset covers all budgets with difficulty, risk and eligibility fields, with a held-out kappa of 0.63 (Objective 2); the router retained 91.8% of A's successes offline and 93% live, above the 90% bar (Objective 3); every claimed difference rests on a corrected paired test (Objective 4); and the repository and appendices document the pipeline (Objective 5). Chapter 6 examines what these results can bear.

5 Use of Generative AI

Generative AI was used in this project in the form of Anthropic's Claude (Anthropic, 2026). Its principal use was assistance with code during development, which this chapter describes and which is logged in AI_Usage.md in the repository. All prompts are reproduced in Appendix D.

5.1 Declaration of use

Code. Five functions in the annotation and analysis scripts were generated by Claude Opus 5 from my prompts: the gold-set consistency check in label_tool.py, the classifier self-test in classify_tasks.py, the Wilson interval and exact McNemar test in shared.py, the Bonferroni correction in compare_abc.py, and the two paired scheme comparisons in analyse_schemes.py (Table 5.1; prompts in Appendix D.1). Every other line of code, the experiments, the collected data, the hand-labelled gold set, the trained router and the analyses are my own work.

5.2 How AI was used in code development

The pattern was the same for each of the five uses. I described the function I needed, its inputs and the property it had to have in a single prompt; Claude returned a function; I read it, ran it and integrated it only when it did what the prompt asked. The prompts were specific because the point was to implement a method I had already chosen, not to ask which method to use: the interval "needs to work well when the success rate is very low or zero"; the paired test "should only look at the tasks where one strategy succeeded and the other failed". Table 5.1 summarises the five uses.

Table 5.1 The 5 AI-generated functions, the line they occupy in the repository and what each taught me.

Script	What was asked for	Lines	What I learnt
label_tool.py	A check that the hand-labelled gold-set CSV is consistent with the unedited gold-set JSONL	206-250	My hand labels had to agree with each other before they could serve as ground truth
classify_tasks.py	A self-test over a supplied list of task intents	263-345	The same pattern was reused for the self-test in flag_episodes.py
shared.py	A 95% interval for a success rate that behaves at zero successes, and a paired test of two strategies on the same tasks using only the discordant tasks	75-84, 90-97	The normal-approximation interval fails at zero successes, common at 2k to 4k; Wilson stays inside [0, 1]. Only discordant pairs carry evidence, so independent-sample tests would be wrong
compare_abc.py	A correction for the many McNemar tests run across strategy pairs and difficulty tiers	123-150, 291-313	The multiple-comparisons problem and the 0.05/3 and 0.05/9 thresholds; Bonferroni is conservative, so A against C ($p = 0.065$) is reported as no detectable difference, not equality

analyse_schemes.py	Two task-paired comparisons: scheme against scheme within a strategy (success and tokens), and strategy against strategy under one scheme	114-147, 150-171	Binary success needs McNemar on the discordant tasks; continuous token differences need the Wilcoxon signed-rank test, which does not assume normality

5.3 How outputs were evaluated before integration

Each generated function went through three tests before being added to the repository. First, it was tested line by line to see if it implemented the correct definition for the corresponding function (i.e., the Wilson formula (Equation 3.5), the number of discordant pairs for the exact McNemar test (Equation 3.6), and the Bonferroni threshold). Then, the function was tested on test cases where one would expect certain results. For example, for a case with zero successes the interval should start with 0, not with some number less than 0. Also, two strategies that are identical should give p = 1. Finally, a case with a pair of cells that are discordant on one side only should give p = 2^k * 0.5^k, where k is the number of cells that are discordant, and this value should be less than or equal to 1. Third, the functions were tested as part of the overall test of the pipeline. This includes a self-test (Section 3.6) done by classify_tasks.py as well as a synthetic-episode test (Section 3.6) of the flag_episodes.py function. Also, the functions were tested as part of a more overall test and their printouts were checked for consistency.

5.4 Where AI was wrong

None of the five generated functions needed correction after review, and the self-tests passed first time, so the code record in Appendix D.1 shows no mistakes.

5.5 Reflection

This AI helps in best way possible as translator from programming method to its implementation. It is only as good as input it receives. For reporting results from four substantive lessons in the four statistical lessons outlined in Table 5.1, I used Wilson intervals as an alternative to normal approximations, paired tests as opposed to independent tests, corrected thresholds, and different tests for binary and for continuous outcomes. Having specified, examined, and tested the implementation as opposed to just calling a library function to implement it was very beneficial to my understanding of the four methodological lessons. Assistance was kept safe by not including any content in the repository or report that could not be traced back to a definition, a test, or a file with my own output from running a test. In future such work, I would like to have test cases for each function, as opposed to having to write them after the function was written.

6 Discussion

6.1 The cost of orchestration

The central result is arithmetic. Strategy B step costs 2.8x a Strategy A step because three calls re-send the same page and carry 2,050 instruction tokens, so under a fixed budget the pipeline pays its overhead before it can earn anything back. At 2k and 4k it cannot take a single step; from 8k to 32k it trails the single agent by 7 to 13 points; only at 64k, where the budget stops binding for both, do the curves meet. Matching success levels needed two to four times the budget, and on tasks both solved the pipeline spent 2.9 times as much. This is the compute-matched lesson of Wang et al. (2024b) reproduced on a stateful web benchmark, and it agrees with the risk-averse policies BAMAS learned under tight budgets (Yang, L. et al., 2025): elaborate topologies pay only where budget is slack.

6.2 What review is worth in a fixed pipeline

The Critic consumed 31.5% of the pipeline's spend and changed 5.1% of the actions it saw; 95% of B's successes involved no revision, and episodes containing one succeeded less often. A reviewer that reads the same page as the executor has little independent information, so its approvals are expensive confirmations. The one signal that predicted a useful revision was a preceding action error, where revisions were 2.6 times more likely, which suggests review should be conditional on evidence the actor lacks rather than paid on every step. The pipeline's fifteen unique solutions came from the Planner: eleven succeeded without any revision, on tasks where the single agent looped through the same pages. Decomposition helps where the agent is lost; review rarely helps where it is wrong. This echoes the finding of Zhang et al. (2024) that most inter-agent communication is redundant: here the review channel carried almost nothing.

6.3 Learned routing against the alternatives

In terms of a straightforward insight into how this worked, the mode head chose the best pipeline 5 times out of 2,604 events. In terms of the training data, there were 24 events where only B succeeded to complete the task versus 192 events where only A succeeded to complete the task. So, the policy just must infer that we don’t need to orchestrate the tasks for this decision-making process. In terms of the efficiency gain, the stop head declined to explore many unpromising cells in the state space, in 77% of cases at a threshold of 2k tokens and in almost all cases before taking an action. That leaves us with a position somewhere between an always-A type of strategy and the hindsight oracle of a router, which in this case’s experiments was able to observe savings of 87%. However, the current strategy’s costs are 36 tasks that A completed successfully while C declined to complete them for some reason or other. This router predominantly decides whether to execute a task in BAVT (Li et al., 2026), which can adapt to diminishing resources of all kinds, as a function of this router.

6.4 Confidence, validity and generalisability

The major threat to the validity of internal findings for this study is the use of a single backbone model to test all three sites, collecting only one example per cell and running no repeats to assess variance. The difference in A's accounting for a one-step overshoot to that for a two-step overshoot is bounded. The difficulty labels were assigned with a held-out kappa of 0.63 (the weakest dimension was pages at 0.43). For RQ3, a sample of 100 tasks was chosen that were heavily centred on shopping. The nine-point scales were not sufficient to pick up any scheme effects that were observed. The router was trained and tested on the same task population. Although the use of folds to test task generalization effectively prevents task leakage, distributional shift is not addressed. The major finding of the study, that any pipeline that re-sends the page will result in a cost amplification that is proportional to the call count of that scheme, does not generalize to the magnitudes of difference.

6.5 Implications and recommendations

Following from these recommendations, under a fixed budget it is best to start with a single agent, add a planner for loops, and add a critic only after an action error. Further, rather than distributing tokens in a task and then trying to complete the task, it is far more important to determine whether one should do a task at all, and none of the current allocation schemes improve success rate. For the evaluator, cost-matched comparisons (e.g. using paired tests, decontamination) should be the norm for stateful cases (e.g. benchmarks with memory), since unconstrained comparison favours the highest resource allocator. The same would hold for a budget-aware design like BAMAS or AgentBalance (Section 2.4); one should assess savings against a budget-matched single agent, not a whole unlimited pipeline.

7 Evaluation, Reflections and Conclusions

7.1 Evaluations against the objectives

All five objectives met their tests (Section 1.3). The harness ran all three strategies end to end with one schema (Objective 1). The dataset covers every cell with difficulty, risk and eligibility labels, and the annotator agreed with held-out hand labels at kappa 0.63 (Objective 2). The router retained 91.8% of A's successes offline and 93% live, above the 90% bar, although its mode head proved almost inert (Objective 3). Every claimed difference rest on a corrected paired test (Objective 4), and the repository and appendices make the pipeline reproducible (Objective 5).

7.2 Evaluation of the methods and the plan

The shared harness, the decontamination rule and the paired design worked as intended and are why the comparisons can be trusted. Three things went less well. The proposal's four-budget ladder was too narrow and was extended once B's behaviour was seen; the second instruction set for B cost a full collection before its fault was found; and the plan slipped, with collection running from mid-July to late September. Cost bounded the scope: the EC2 instance cost about £250 from July to October, near £100 a month during the main collections, plus OpenAI API charges for roughly 116 million tokens (£40). That budget allowed one model, three sites and one run per cell, so run-to-run variance is unmeasured.

7.3 Reflections on the project process

A daily journal and resumable, append-only collection scripts turned crashes and API failures into hours lost rather than days. The most important lesson I learned was to verify the instructions given before running a batch of scripts. This is now part of the start-up procedure for a project of this nature. I had also underestimated the amount of time verification would take and the checks in Section 3.6 were added after each fault. It would have been more cost effective to include them in the first place. Running the cloud instance for three months also taught me an important lesson. The environment is a critical component of the experiment and must be considered. On occasion I had to re-build instances that had crashed, deal with timeouts and even patch the evaluator scripts to get stable results.

7.4 Contributions

The project contributes a controlled, cost-matched comparison of orchestration strategies on WebArena; an annotated dataset of 9,450 episodes with difficulty, risk and eligibility labels; a decontamination procedure for stateful benchmarks; a trained offline router with its training data; evidence that a Critic approving 95% of actions is overhead rather than safety; and a kappa-validated LLM difficulty-annotation protocol reusable for any WebArena study.

7.5 Further work

There are several obvious extensions, including a secondary backbone for the residual WebArena sites, repeated iterations of variance estimation, a conditional Critic for action errors, and a stop head that is trained using on-policy data for the recovery of the 36 tasks that it had erroneously declined. The per-step role router that was proposed is even more ambitious, and it would require training data that invoke the different roles in very different patterns, than the logged A and B episodes in this paper provide.

7.6 Conclusions

Learning to orchestrate within a fixed token budget is much harder than acquiring a fixed pipeline to spend. A learned router can generally keep as many successes as the single best agent within budget, but with the added value of rejecting opportunities that it wouldn’t win anyway. Within an episode, the differences in how tokens are allocated to individual acts are minor compared to the major difference in whether an episode is allocated at all.

Glossary

Term	Definition
Amazon EC2	Amazon's cloud service renting virtual machines by the hour. The environment ran on one t3a.xlarge instance (4 vCPU, 16 GiB RAM, 1,000 GiB disk).
AMI (Amazon Machine Image)	A snapshot of an operating system with software pre-installed, from which an EC2 instance boots into a known configuration; WebArena's maintainers publish one with the benchmark sites installed.
BrowserGym	The library that presents each WebArena task as a gym environment: reset() loads the task's start page and step(action) executes an action, returning the next observation and the evaluator's reward.
Docker container	An isolated, reproducible software environment. Each WebArena website runs as one container, rebuilt from the original image before each strategy's collection.
tmux	A terminal multiplexer that keeps a remote session running if the SSH connection drops; all long collections ran inside it.
Fuzzy-match evaluator	WebArena's checker for free-text answers: it calls an LLM to judge whether the agent's submitted answer matches the reference answer.
Idempotent task	A task that changes site state but converges to the same state however often it is repeated (e.g. adding an item to a wishlist); low contamination risk.
Non-idempotent task	A task whose repetitions leave the site progressively changed (orders, posts, edits, deletions); the highest contamination risk.
Accessibility tree	The browser's structured text description of a page's visible elements. Flattened and filtered to elements that carry an identifier, it is the observation the agents receive instead of raw HTML or pixels.
Action error	A step whose chosen action fails to execute, for example because it names an element not on the page; four consecutive action errors end the episode (a guard).
Confidence interval (CI)	A range that would contain the true value in 95% of repeated experiments; every success rate in Chapter 4 is reported with one.
Large language model (LLM)	A neural network trained on text to generate text. Used here as the agents' backbone (GPT-5 mini), as the difficulty annotator, and inside WebArena's fuzzy-match evaluator.
MLP (multilayer perceptron)	A small feed-forward neural network. Both router heads are MLPs with two hidden layers of 64 units.
p-value	The probability of observing a difference at least as large as the one measured if the two strategies were in fact equally good; values below the Bonferroni-corrected threshold are reported as significant.
Advantage-weighted regression (AWR)	The offline reinforcement-learning method used to train the router: logged decisions are imitated with a weight that grows with how much better than average their outcome was.
Bonferroni correction	Dividing the 0.05 significance threshold by the number of tests run together (e.g. 0.05/3 for three strategy pairs), so that running many tests does not produce spurious “significant” results by chance.
Cohen's kappa (κ)	Agreement between two raters corrected for the agreement expected by chance. The quadratic-weighted form penalises large disagreements more

	than near-misses, suiting ordinal scores; 0.61–0.80 is “substantial” on the Landis and Koch scale.
McNemar test (exact)	A paired test for binary outcomes. It uses only the discordant cells those where exactly one of the two strategies succeeded since only they carry evidence of a difference.
Wilcoxon signed-rank test	A paired test for continuous outcomes (here token spends) that does not assume the differences are normally distributed.
Wilson interval	A confidence interval for a proportion that stays within [0, 1] and behaves sensibly at zero successes, where the usual normal approximation fails.
Token	The unit in which language models read and write text, roughly four characters of English. Tokens are the unit of cost, and of the budgets.
Reasoning effort	A GPT-5 mini setting controlling how many hidden reasoning tokens the model may spend before answering. Pinned to low throughout, except for the designated high-effort steps of the RQ3 schemes.
Reasoning tokens	Hidden tokens a reasoning model generates before its visible answer. They are billed as output tokens and counted in full against the budget.
Budget ladder	The six budget levels (2k, 4k, 8k, 16k, 32k, 64k) at which every task was attempted by every strategy.
Offline reinforcement learning	Learning a decision policy purely from previously logged data, with no interaction with the environment during training; how Strategy C's router was trained from A's and B's episodes.
Oracle (hindsight)	A reference policy that, knowing every episode's outcome in advance, stops each failure before it spends anything. It is unattainable in practice and marks the ceiling for what a stop decision could save.

References

Amazon Web Services (2025) Amazon EC2 user guide. Available at: https://docs.aws.amazon.com/ec2/

Cai, S., Ning, Y. and Liu, H. (2025) 'AgentBalance: backbone-then-topology design for cost-effective multi-agent systems under budget constraints', arXiv preprint.

Chen, Y., Chen, J., He, C. et al. (2026) 'Token economics for LLM agents: a dual-view study from computing and economics', arXiv preprint.

Cohen, J. (1960) 'A coefficient of agreement for nominal scales', Educational and Psychological Measurement, 20(1), pp. 37-46.

Cohen, J. (1968) 'Weighted kappa: nominal scale agreement with provision for scaled disagreement or partial credit', Psychological Bulletin, 70(4), pp. 213-220.

Docker Inc. (2025) Docker documentation. Available at: https://docs.docker.com/

Dunn, O.J. (1961) 'Multiple comparisons among means', Journal of the American Statistical Association, 56(293), pp. 52-64.

Efron, B. (1979) 'Bootstrap methods: another look at the jackknife', The Annals of Statistics, 7(1), pp. 1-26.

Harris, C.R. et al. (2020) 'Array programming with NumPy', Nature, 585, pp. 357-362.

Hunter, J.D. (2007) 'Matplotlib: a 2D graphics environment', Computing in Science & Engineering, 9(3), pp. 90-95.

Kingma, D.P. and Ba, J. (2015) 'Adam: a method for stochastic optimization', in Proceedings of the 3rd International Conference on Learning Representations (ICLR 2015). San Diego, CA. arXiv:1412.6980.

Landis, J.R. and Koch, G.G. (1977) 'The measurement of observer agreement for categorical data', Biometrics, 33(1), pp. 159-174.

Le Sellier De Chezelles, T. et al. (2024) 'The BrowserGym ecosystem for web agent research', arXiv preprint arXiv:2412.05467.

Levine, S., Kumar, A., Tucker, G. and Fu, J. (2020) 'Offline reinforcement learning: tutorial, review, and perspectives on open problems', arXiv preprint arXiv:2005.01643.

Li, Y., Deng, W., Li, J. and Li, X. (2026) 'Spend less, reason better: budget-aware value tree search for LLM agents', arXiv preprint arXiv:2603.12634.

McKinney, W. (2010) 'Data structures for statistical computing in Python', in Proceedings of the 9th Python in Science Conference (SciPy 2010). Austin, TX, pp. 56-61.

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', Psychometrika, 12(2), pp. 153-157.

Microsoft (2024) Playwright for Python documentation. Available at: https://playwright.dev/python/

Mohammadi, M., Li, Y., Lo, J. and Yip, W. (2025) 'Evaluation and benchmarking of LLM agents: a survey', in Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25). Toronto, Canada, 3-7 August. New York: ACM. doi: 10.1145/3711896.3736570.

OpenAI (2024) GPT-4o mini: advancing cost-efficient intelligence. Available at: https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

OpenAI (2025) GPT-5 mini: model documentation. Available at: https://platform.openai.com/docs/models/gpt-5-mini

Paszke, A. et al. (2019) 'PyTorch: an imperative style, high-performance deep learning library', in Advances in Neural Information Processing Systems 32 (NeurIPS 2019). Vancouver, Canada, pp. 8024-8035.

Pearson, K. (1900) 'On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling', Philosophical Magazine, 50(302), pp. 157-175.

Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825-2830.

Peng, X.B., Kumar, A., Zhang, G. and Levine, S. (2019) 'Advantage-weighted regression: simple and scalable off-policy reinforcement learning', arXiv preprint arXiv:1910.00177.

Pydantic (2025) Pydantic AI documentation. Available at: https://ai.pydantic.dev/

Salim, M., Latendresse, J., Khatoonabadi, S. and Shihab, E. (2026) 'Tokenomics: quantifying where tokens are used in agentic software engineering', in Proceedings of the International Conference on Mining Software Repositories (MSR '26). Rio de Janeiro, Brazil. New York: ACM.

Shi, T., Karpathy, A., Fan, L., Hernandez, J. and Liang, P. (2017) 'World of Bits: an open-domain platform for web-based agents', in Proceedings of the 34th International Conference on Machine Learning (ICML 2017). Sydney, Australia: PMLR, pp. 3135-3144.

Virtanen, P. et al. (2020) 'SciPy 1.0: fundamental algorithms for scientific computing in Python', Nature Methods, 17, pp. 261-272.

Wang, J., Jain, S., Zhang, D., Ray, B., Kumar, V. and Athiwaratkun, B. (2024b) 'Reasoning in token economies: budget-aware evaluation of LLM reasoning strategies', arXiv preprint arXiv:2406.06461.

Wang, J., Wang, J., Athiwaratkun, B., Zhang, C. and Zou, J. (2024a) 'Mixture-of-Agents enhances large language model capabilities', arXiv preprint arXiv:2406.04692.

Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', Biometrics Bulletin, 1(6), pp. 80-83.

Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', Journal of the American Statistical Association, 22(158), pp. 209-212.

Xu, F.F. et al. (2024) 'TheAgentCompany: benchmarking LLM agents on consequential real world tasks', arXiv preprint arXiv:2412.14161.

Yang, B., He, X., Gao, H., Cao, Y., Li, X. and Hsu, D. (2025) 'CodeAgents: a token-efficient framework for codified multi-agent reasoning in LLMs', arXiv preprint arXiv:2507.03254.

Yang, L., Luo, J., Liu, X., Lou, Y. and Chen, Z. (2025) 'BAMAS: structuring budget-aware multi-agent systems', arXiv preprint arXiv:2511.21572.

Zhang, G., Yue, Y., Li, Z. et al. (2024) 'Cut the crap: an economical communication pipeline for LLM-based multi-agent systems', arXiv preprint arXiv:2410.02506.

Zhou, S., Xu, F.F., Zhu, H. et al. (2023) 'WebArena: a realistic web environment for building autonomous agents', arXiv preprint arXiv:2307.13854. Published at the International Conference on Learning Representations (ICLR 2024).

Appendix A – Full Results Tables

A.1 Strategy A – Complete per-budget, per-site, and per-tier breakdowns

The tables in this section are the output of analyse_a.py run on final_episodes_a.jsonl (command in Appendix B.2). 171 of 475 tasks were solved at least once; tasks first solved at each budget: 6, 18, 30, 43, 42 and 32. Token distribution: 25th percentile 3,628, median 7,674, 75th percentile 15,780, maximum 65,293; median of successful episodes 6,570 (n = 383). Per-step cost by site (median): reddit 2,278, shopping 2,338, shopping_admin 2,545; overall median 2,480, mean 2,484.

Table A.1.1 Success by budget level. Task success counts a task once per budget; median tokens and steps are over all episodes at that budget.

Budget	Episodes	Successes	Episode success	Median tokens	Median steps
2k	442	6	1.36%	2,382	1
4k	437	24	5.49%	4,494	2
8k	433	54	12.47%	7,747	3
16k	450	95	21.11%	15,157	6
32k	442	104	23.53%	17,676	7
64k	423	100	23.64%	17,391	7

Table A.1.2 Success by site, pooled over budgets

Site	Episodes	Episode Success	Tasks solved / Total	Task Success	Median tokens	Median steps
Reddit	549	10.38%	45 / 106	42.45%	8,711	4
Shopping	1,051	19.41%	65 / 187	34.76%	7,446	3
Shopping_admin	1,027	11.88%	61 / 182	33.52%	7,813	3

Table A.1.3 Termination reasons over all eligible episodes

Termination reason	Episodes	Share
safety_token_cap	1,039	40%
budget_would_exceed	439	17%
env_terminated	422	16%
success	383	15%
navigation_cycle	325	12%
repeated_action_failure	13	0%
max_steps	6	0%

Table A.1.4 Episode success rate by difficulty tier and budget

Tier	2k	4k	8k	16k	32k	64k
Easy	1.47%	4.44%	13.74%	27.86%	32.14%	30.08%
Medium	1.67%	5.96%	12.71%	19.75%	20.00%	22.65%
Hard	0.00%	5.97%	9.09%	11.94%	17.91%	15.15%

Table A.1.5 Budget as the binding constraint and budget utilisation, by budget level

Budget	Failures	Budget-driven failures	Utilisation, all episodes	Utilisation, successes	Utilisation, failures
2k	436	436 (100%)	119%	99%	119%
4k	413	404 (98%)	112%	53%	113%
8k	379	328 (87%)	97%	49%	98%
16k	355	233 (66%)	95%	48%	99%
32k	338	72 (21%)	55%	25%	64%
64k	323	5 (2%)	27%	12%	32%

Wrong-answer episodes: 422 (16% of episodes), median 9,144 tokens and 4 steps before answering; by budget 9 (4k), 42 (8k), 100 (16k), 131 (32k), 140 (64k); by site reddit 59, shopping 180, shopping_admin 183.

A.2 Strategy B – Complete per-budget, per-site, and per-tier breakdowns

The tables in this section are the output of analyse_b.py run on final_episodes_b.jsonl (command in Appendix B.2). 120 of 475 tasks were solved at least once; tasks first solved at each budget: 26 (8k), 11 (16k), 26 (32k), 57 (64k). Token distribution: 25th percentile 0, median 7,241, 75th percentile 27,580, maximum 68,681; median of successful episodes 11,994 (n = 226). Per-step cost by site (median): reddit 5,836, shopping 6,395, shopping_admin 7,365; overall median 6,967, mean 6,883.

Table A.2.1 Success by budget level. Task success counts a task once per budget; median tokens and steps are over all episodes at that budget

Budget	Episodes	Successes	Episode success	Median tokens	Median steps
2k	448	0	0.00%	0	0
4k	451	0	0.00%	0	0
8k	456	26	5.70%	6,050	1
16k	449	35	7.80%	13,691	2
32k	454	60	13.22%	29,211	4
64k	465	105	22.58%	58,527	8

Table A.2.2 Success by site, pooled over budgets

Site	Episodes	Episode success	Tasks solved / total	Task success	Median tokens	Median steps
reddit	600	3.83%	18 / 106	16.98%	9,442	1
shopping	1,069	11.97%	55 / 187	29.41%	6,093	1
shopping_admin	1,054	7.12%	47 / 182	25.82%	7,424	1

Table A.2.3 Termination reasons over all eligible episodes

Termination reason	Episodes	Share
budget_would_exceed	2,218	81%
success	226	8%
safety_token_cap	137	5%
env_terminated	83	3%
navigation_cycle	57	2%
repeated_action_failure	2	0%
budget_would_exceed	2,218	81%

Table A.2.4 Episode success rate by difficulty tier and budget

Tier	2k	4k	8k	16k	32k	64k
Easy	0.00%	0.00%	4.90%	9.42%	18.57%	31.29%
Medium	0.00%	0.00%	6.56%	7.44%	11.84%	20.88%
Hard	0.00%	0.00%	4.35%	5.80%	7.25%	10.14%

Table A.2.5 Critic calls and revisions by budget (analyse_b.py)

Budget	Critic calls	Revisions	Revision rate
2k	0	0	-
4k	0	0	-
8k	415	4	1.0%
16k	886	15	1.7%
32k	1,761	70	4.0%
64k	3,201	233	7.3%
All	6,263	322	5.1%

Table A.2.6 Token cost by role over all eligible Strategy B episodes (analyse_b.py)

Role	Tokens	Share	Mean tokens per logical step
Planner	16,667,182	38.1%	2,661
Executor	13,309,331	30.4%	2,125
Critic	13,764,452	31.5%	2,198
Total	43,740,965	100.0%	6,984

Revision profile: 231 episodes (8.5%) contained a revision; success rate 4.8% with a revision against 8.6% without; 20 revisions (6%) followed an action error against a 2% base rate of post-error calls (lift 2.56); median step at revision 4, median budget remaining 31%; 210 of 322 revised actions (65%) executed without error; 16 revisions (5%) occurred in successful episodes; 215 of 226 successes had no revision. Approval-only calls: 5,941 of 6,263 (95%), costing 13,004,279 tokens, 29.7% of all Strategy B spend.

A.3 Strategy C - Complete per-budget, per-site, and per-tier breakdowns

The tables in this section are the output of analyse_c.py run on final_episodes_c.jsonl (command in Appendix B.2). 157 of 475 tasks were solved at least once; tasks first solved at each budget: 6, 20, 23, 30, 46 and 32. Token distribution: 25th percentile 0, median 6,084, 75th percentile 14,317, maximum 64,650; median of successful episodes 5,901 (n = 360). Per-step cost (median): reddit 2,375, shopping 2,363, shopping_admin 2,589; overall median 2,490, mean 2,513.

Table A.3.1 Success by budget level. Task success counts a task once per budget; median tokens and steps are over all episodes at that budget

Budget	Episodes	Successes	Episode success	Median tokens	Median steps
2k	428	6	1.40%	0	0
4k	428	26	6.07%	2,052	1
8k	427	47	11.01%	7,434	3
16k	439	76	17.31%	12,424	5
32k	446	105	23.54%	14,920	6
64k	436	100	22.94%	15,518	6

Table A.3.2 Success by site, pooled over budgets

Site	Episodes	Episode success	Tasks solved / total	Task success	Median tokens	Median steps
reddit	525	10.10%	41 / 106	38.68%	7,858	3
shopping	1,067	17.81%	58 / 187	31.02%	5,024	2
shopping_admin	1,012	11.56%	58 / 182	31.87%	5,146	2

Table A.3.3 Termination reasons over all eligible episodes

Termination reason	Episodes	Share
router_stop	838	32%
safety_token_cap	475	18%
env_terminated	408	16%
success	360	14%
navigation_cycle	290	11%
budget_would_exceed	230	9%
max_steps	2	0%
repeated_action_failure	1	0%

Table A.3.4 Episode success rate by difficulty tier and budget

Tier	2k	4k	8k	16k	32k	64k
Easy	0.00%	6.11%	12.50%	21.80%	28.37%	28.79%
Medium	2.14%	6.06%	11.16%	16.25%	22.36%	21.52%
Hard	1.52%	6.06%	7.58%	12.12%	17.65%	16.42%

Table A.3.5 Router stops by budget, with the success rate over all episodes and over attempted episodes only (analyse_c.py)

Budget	Episodes	Router stops	Stop rate	Success, all	Attempted	Success, attempted	Mean tokens	Cycle-mode share
2k	428	331	77.3 %	1.4%	97	6.2%	532	0.0%
4k	428	235	54.9 %	6.1%	193	13.5%	2,035	0.0%
8k	427	115	26.9 %	11.0%	312	15.1%	5,562	0.0%
16k	439	70	15.9 %	17.3%	369	20.6%	10,470	0.0%
32k	446	50	11.2 %	23.5%	396	26.5%	15,785	0.7%
64k	436	37	8.5%	22.9%	399	25.1%	17,687	0.5%

Table A.3.6 Router stops by difficulty tier (analyse_c.py)

Tier	Episodes	Stop rate	Success, all	Success, attempted	Gap
Easy	793	27.0%	16.5%	22.6%	6.1 pp
Medium	1,412	31.1%	13.3%	19.3%	6.0 pp
Hard	399	46.4%	10.3%	19.2%	8.9 pp

Of the 838 router stops, 703 occurred at step 0 (before any action), 76 at step 1, 14 at step 2, 14 at step 3 and 31 later. The mode head chose the pipeline (cycle mode) in 5 episodes: task 459 at 64k (success), 460 at 32k (success), 462 at 32k and 64k (failures) and 463 at 32k (success).

A.4 Strategy Comparisons - Strategy comparisons: paired tests, equivalent budgets and failure modes

The tables in this section are the output of compare_abc.py run on the three final episode files (the full printout is held in the project as additional materials/diagrams/Batch Outputs/Comparisons/compare_abc_output.txt).

Table A.4.1 Cell sizes and success by budget for the three strategies

Budget	A episodes	A success	B episodes	B success	C episodes	C success
2k	442	1.4%	448	0.0%	428	1.4%
4k	437	5.5%	451	0.0%	428	6.1%
8k	433	12.5%	456	5.7%	427	11.0%
16k	450	21.1%	449	7.8%	439	17.3%
32k	442	23.5%	454	13.2%	446	23.5%
64k	423	23.6%	465	22.6%	436	22.9%

Table A.4.2 Paired McNemar tests by budget: number of paired cells, successes by both, by one strategy only, and the exact p-value

Pair	Budget	Pairs	Both	First only	Second only	p
A vs B	2k	430	0	6	0	0.0312
	4k	425	0	24	0	< 0.0001
	8k	424	24	30	0	< 0.0001
	16k	433	34	52	0	< 0.0001
	32k	428	49	47	7	< 0.0001
	64k	420	66	33	17	0.0328
	all	2,560	173	192	24	< 0.0001
A vs C	2k	413	2	4	4	1.0000
	4k	411	24	0	1	1.0000
	8k	410	44	9	3	0.1460
	16k	428	62	23	13	0.1325
	32k	429	73	23	24	1.0000
	64k	409	70	24	15	0.1996
	all	2,500	275	83	60	0.0654
B vs C	2k	418	0	0	6	0.0312
	4k	418	0	0	25	< 0.0001
	8k	421	22	2	25	< 0.0001
	16k	429	32	3	42	< 0.0001
	32k	436	49	6	52	< 0.0001
	64k	431	64	23	33	0.2288
	all	2,553	167	34	183	< 0.0001

Table A.4.3 Equivalent budget relative to Strategy A: the smallest A budget whose success rate matches each B and C budget

Strategy and budget	Success	Matches A at	A success	Budget multiple
B 2k	0.0%	2k	1.4%	1.0x
B 4k	0.0%	2k	1.4%	2.0x
B 8k	5.7%	4k	5.5%	2.0x
B 16k	7.8%	4k	5.5%	4.0x
B 32k	13.2%	8k	12.5%	4.0x
B 64k	22.6%	32k	23.5%	2.0x
C 2k	1.4%	2k	1.4%	1.0x
C 4k	6.1%	4k	5.5%	1.0x
C 8k	11.0%	8k	12.5%	1.0x
C 16k	17.3%	16k	21.1%	1.0x
C 32k	23.5%	32k	23.5%	1.0x
C 64k	22.9%	32k	23.5%	2.0x

Cost ratio on shared successes (same task and budget solved by both): B/A median 2.90, mean 3.27, range 0.29 to 19.46 over 105 tasks; C/A median 1.00, mean 1.13, range 0.19 to 8.13 over 135 tasks.

Table A.4.4 Task-level overlap: tasks solved at any budget

Solved by	Tasks
A, B and C	95
A and B only	10
A and C only	40
A only	26
B and C only	5
B only	10
C only	17
None	272

Table A.4.5 The fifteen tasks Strategy B solved that Strategy A never solved

Task	Budget	B steps	Critic revisions	A's termination on the task
78	64k	5	0	navigation_cycle
96	64k	4	0	env_terminated
117	64k	5	0	env_terminated
195	64k	7	0	env_terminated
240	64k	7	1	navigation_cycle
347	32k	3	0	env_terminated
459	32k	3	1	max_steps
460	64k	4	0	max_steps
461	32k	3	0	max_steps
462	64k	3	0	max_steps
463	32k	3	2	max_steps
515	64k	4	0	navigation_cycle
519	64k	4	1	navigation_cycle
603	64k	10	0	navigation_cycle
651	64k	5	0	navigation_cycle

Tasks C solved that A never did: 22 (including 4, 31, 79, 114, 159, 346, 408, 459, 460, 463, 515, 574, 575, 595, 603). Tasks A solved that C never did: 36; C's episodes on them ended budget exhausted 72, router stop 63, wrong answer 43, stuck (guards) 37, other 1.

Table A.4.6 Failure mode shift: share of each strategy's failures by cause

Strategy	Failures	Wrong answer	Budget exhausted	Stuck (guards)	Router stop	Other
A	2,244	422 (19%)	1,478 (66%)	338 (15%)	-	6 (0%)
B	2,497	83 (3%)	2,355 (94%)	59 (2%)	-	0
C	2,244	408 (18%)	705 (31%)	291 (13%)	838 (37%)	2 (0%)

Table A.4.7 Success rate by tier and budget for each strategy

Strategy, tier	2k	4k	8k	16k	32k	64k
A Easy	1.5%	4.4%	13.7%	27.9%	32.1%	30.1%
A Medium	1.7%	6.0%	12.7%	19.8%	20.0%	22.6%
A Hard	0.0%	6.0%	9.1%	11.9%	17.9%	15.2%
B Easy	0.0%	0.0%	4.9%	9.4%	18.6%	31.3%
B Medium	0.0%	0.0%	6.6%	7.4%	11.8%	20.9%
B Hard	0.0%	0.0%	4.3%	5.8%	7.2%	10.1%
C Easy	0.0%	6.1%	12.5%	21.8%	28.4%	28.8%
C Medium	2.1%	6.1%	11.2%	16.3%	22.4%	21.5%
C Hard	1.5%	6.1%	7.6%	12.1%	17.6%	16.4%

Table A.4.8 Paired McNemar tests within each difficulty tier

Pair	Tier	Pairs	First success	Second success	First only	Second only	p
A vs B	Easy	755	17.7%	9.5%	73	11	< 0.0001
	Medium	1,405	13.6%	7.5%	96	11	< 0.0001
	Hard	400	10.0%	4.8%	23	2	< 0.0001
A vs C	Easy	731	18.1%	16.0%	35	20	0.0581
	Medium	1,374	13.7%	13.1%	44	36	0.4340
	Hard	395	9.6%	9.6%	4	4	1.0000
B vs C	Easy	751	9.9%	16.5%	17	67	< 0.0001
	Medium	1,403	7.7%	13.2%	15	92	< 0.0001
	Hard	399	4.8%	10.3%	2	24	< 0.0001

Table A.4.9 Difficulty sensitivity: drop from Easy to Hard and the Hard/Easy ratio, with task-level bootstrap 95% intervals, and the paired differences between strategies

Strategy or contrast	Drop Easy to Hard (pp) [CI]	Hard / Easy ratio [CI]
A	8.3 [0.89, 15.38]	0.55 [0.25, 0.94]
B	6.3 [1.48, 10.64]	0.42 [0.13, 0.86]
C	6.2 [-1.31, 13.42]	0.62 [0.28, 1.09]
B minus A	-	-0.13 [-0.35, +0.08] (no clear difference)
C minus A	-	+0.07 [-0.03, +0.23] (no clear difference)
C minus B	-	+0.20 [+0.01, +0.44] (differs)

Table A.4.10 Failure modes by difficulty tier, with the Pearson chi-square test of independence between tier and failure cause

Strategy, tier	Failures	Wrong answer	Budget exhausted	Stuck (guards)	Router stop	Chi-square
A Easy	658	15%	69%	15%	-	chi2 = 19.8, dof = 6, p = 0.0029
A Medium	1,226	20%	65%	15%	-	-
A Hard	360	23%	63%	14%	-	-
B Easy	750	4%	93%	3%	-	chi2 = 9.5, dof = 4, p = 0.0499
B Medium	1,352	4%	95%	2%	-	-
B Hard	395	2%	96%	3%	-	-
C Easy	662	15%	36%	17%	32%	chi2 = 72.6, dof = 8, p < 0.0001
C Medium	1,224	19%	32%	12%	36%	-
C Hard	358	21%	20%	8%	52%	-

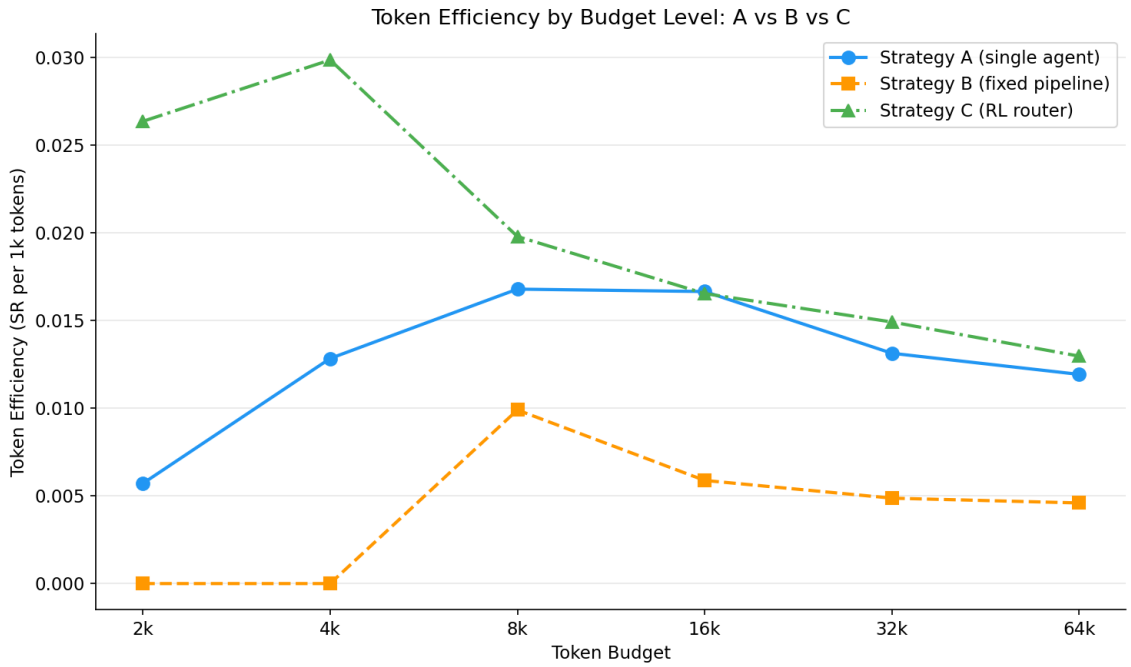


Figure A.4.1 Token efficiency (successes per 1,000 tokens) by budget (compare_abc.py, token_efficiency_a_vs_b_vs_c.png)

A.5 Spending schemes

The tables in this section are the output of analyse_schemes.py run on schemes.jsonl with the three main-grid files supplying the pay-as-you-go reference (100 tasks, 32k budget, 100 episodes in every strategy-scheme cell).

Table A.5.1 RQ3 grid at a 32k budget on 100 read-only tasks: success with Wilson 95% intervals, mean tokens, tokens per success, and the share of steps whose page was trimmed or run at high reasoning effort (analyse_schemes.py)

Strategy	Scheme	Success [CI]	Mean tokens	Tokens per success	Steps trimmed	High-effort steps
A	Pay as you go	22% [15, 31]	15,580	70,820	-	-
	Even	20% [13, 29]	15,435	77,174	0.5%	0%
	Front-loaded	22% [15, 31]	17,071	77,595	0.6%	23%
	Reactive	26% [18, 35]	17,437	67,066	0.4%	30%
B	Pay as you go	12% [7, 20]	27,607	230,060	-	-
	Even	14% [9, 22]	29,321	209,432	52%	0%
	Front-loaded	7% [3, 14]	28,512	407,320	78%	22%
	Reactive	13% [8, 21]	29,619	227,838	49%	43%
C	Pay as you go	22% [15, 31]	15,693	71,333	-	-
	Even	23% [16, 32]	14,708	63,947	0.1%	0%
	Front-loaded	23% [16, 32]	14,598	63,471	0.2%	26%
	Reactive	23% [16, 32]	17,342	75,400	0.3%	32%

Table A.5.2 Effect of each scheme relative to pay-as-you-go, paired bootstrap over tasks (2,000 resamples), 95% intervals (analyse_schemes.py)

Strategy	Scheme	Change in success (pp) [CI]	Change in tokens (%) [CI]
A	Even	-2.0 [-8.0, +3.0]	-0.9 [-11.0, +10.7]
	Front-loaded	0.0 [-6.0, +6.0]	+9.6 [-1.4, +22.2]
	Reactive	+4.0 [-1.0, +9.0]	+11.9 [-0.0, +24.1]
B	Even	+2.0 [-3.0, +7.0]	+6.2 [+2.9, +9.8]
	Front-loaded	-5.0 [-10.0, 0.0]	+3.3 [+0.6, +6.3]
	Reactive	+1.0 [-4.0, +6.0]	+7.3 [+5.2, +9.5]
C	Even	+1.0 [-4.0, +6.0]	-6.3 [-15.1, +3.9]
	Front-loaded	+1.0 [-5.0, +7.0]	-7.0 [-14.7, +1.4]
	Reactive	+1.0 [-5.0, +7.0]	+10.5 [-0.5, +23.7]

Table A.5.3 Scheme against scheme within each strategy, paired on task: exact McNemar on success and Wilcoxon signed-rank on the per-task token difference (threshold 0.0028)

Strategy	Comparison	First success	Second success	First only	Second only	p (success)	Median token difference	p (tokens)
A	Even vs Pay as you go	20%	22%	3	5	0.7266	-68	0.5881
	Front-loaded vs Pay as you go	22%	22%	4	4	1.0000	+1,061	0.0153
	Reactive vs Pay as you go	26%	22%	6	2	0.2891	+1,024	0.0310
	Front-loaded vs Even	22%	20%	6	4	0.7539	+1,196	0.0004
	Reactive vs Even	26%	20%	9	3	0.1460	+807	0.0017
	Reactive vs Front-loaded	26%	22%	8	4	0.3877	-588	0.7104
B	Even vs Pay as you go	14%	12%	4	2	0.6875	+1,855	< 0.0001
	Front-loaded vs Pay as you go	7%	12%	1	6	0.1250	+695	0.1137
	Reactive vs Pay as you go	13%	12%	4	3	1.0000	+1,880	< 0.0001
	Front-loaded vs Even	7%	14%	1	8	0.0391	-1,210	0.0007
	Reactive vs Even	13%	14%	4	5	1.0000	+44	0.6060
	Reactive vs Front-loaded	13%	7%	6	0	0.0312	+1,732	< 0.0001
C	Even vs Pay as you go	23%	22%	4	3	1.0000	-122	0.0961
	Front-loaded vs Pay as you go	23%	22%	6	5	1.0000	+920	0.9671
	Reactive vs Pay as you go	23%	22%	5	4	1.0000	+801	0.0159
	Front-loaded vs Even	23%	23%	6	6	1.0000	+1,070	0.1507
	Reactive vs Even	23%	23%	3	3	1.0000	+1,524	< 0.0001
	Reactive vs Front-loaded	23%	23%	6	6	1.0000	+706	0.0055
	Even vs Pay as you go	23%	22%	4	3	1.0000	-122	0.0961

Table A.5.4 Strategy against strategy under the same scheme, paired on task (exact McNemar, threshold 0.0042)

Scheme	Pair	First success	Second success	First only	Second only	p
Pay as you go	A vs B	22%	12%	10	0	0.0020
	A vs C	22%	22%	5	5	1.0000
	B vs C	12%	22%	2	12	0.0129
Even	A vs B	20%	14%	10	4	0.1796
	A vs C	20%	23%	3	6	0.5078
	B vs C	14%	23%	4	13	0.0490
Front-loaded	A vs B	22%	7%	16	1	0.0003
	A vs C	22%	23%	4	5	1.0000
	B vs C	7%	23%	1	17	0.0001
Reactive	A vs B	26%	13%	17	4	0.0072
	A vs C	26%	23%	4	1	0.3750
	B vs C	13%	23%	3	13	0.0213

Table A.5.5 How episodes ended under each scheme

Strategy	Scheme	Success	Wrong answer	Budget exhausted	Stuck (guards)	Router stop
A	Pay as you go	22%	47%	9%	22%	0%
	Even	20%	49%	10%	21%	0%
	Front-loaded	22%	42%	13%	23%	0%
	Reactive	26%	40%	21%	13%	0%
B	Pay as you go	12%	9%	79%	0%	0%
	Even	14%	6%	79%	1%	0%
	Front-loaded	7%	5%	78%	10%	0%
	Reactive	13%	5%	82%	0%	0%
C	Pay as you go	22%	42%	10%	19%	7%
	Even	23%	42%	7%	19%	9%
	Front-loaded	23%	42%	6%	18%	11%
	Reactive	23%	35%	20%	14%	8%

Table A.5.6 Success rate by difficulty tier under each scheme (descriptive; about 33 tasks per tier)

Strategy	Tier	Pay as you go	Even	Front-loaded	Reactive
A	Easy	39%	30%	39%	42%
	Medium	9%	18%	15%	12%
	Hard	18%	12%	12%	24%
B	Easy	27%	30%	18%	30%
	Medium	6%	12%	3%	9%
	Hard	3%	0%	0%	0%
C	Easy	33%	36%	33%	39%
	Medium	18%	18%	21%	12%
	Hard	15%	15%	15%	18%

Mean steps per episode: A 6.00, 5.86, 6.12, 5.47; B 3.81, 4.53, 4.84, 4.13; C 5.91, 5.66, 5.17, 5.52 (pay-as-you-go, even, front-loaded, reactive).

Appendix B – Source Code

B.1 Repository

https://github.com/Tbeard04/Token-Budget-Orchestration-Strategies-for-Web-Agents

B.2 Repository structure and file inventory

Located in the root folder README.md

B.3 Installation and reproduction guide

Table B.3.1 Software and Infrastructure used in the project

Component	Version	Role	Reference
Amazon EC2 (t3a.xlarge, us-east-2)	-	Hosts the WebArena containers and the agents	Amazon Web Services (2025)
Docker	-	Runs the 3 site containers from the WA image	Docker Inc. (2025
WebArena	2023 release (patched)	Tasks, sites and functional evaluators	Zhou et al. (2023)
BrowserGym, browsergym-webarena	0.14.2	Gym interface, observations & action set	Le Sellier De Chezelles et al (2024)
Playwright (Chromium)	1.44.0	Browser automation under BrowserGym	Microsoft (2024)
PydanticAI	1.5.0	Typed agents, structured outputs and token usage	Pydantic (2025)
OpenAI API; GPI-5 mini & GPT-4o mini	Openai 2.46.0	Agent backbone; answer evaluator	OpenAI (2025); openAI (2024)
Python	3.11 / 3.10	All scripts	-
PyTorch	2.13.0	Router networks and training	Paszke et al. (2019)
Scikit-learn	1.9.1	ROC-AUC metrics for the router	Pedregosa et al. (2011)
NumPy; pandas	2.4.6; 3.0.3	Data handling and tables	Harris et al. (2020); McKinney (2010)
SciPy	-	Wilcoxon and chi-square tests	Virtanen et al. (2020)
Matplotlib	-	All result figures	Hunter (2007)
Tmux; Git/GitHub	-	Consistent remote sessions; version control and public repository	-

B.4 Code Listing

Table B.4.1 Code listing in module order.

Script	Module	Location	Purpose
wa_env.py	WebArena environment	src/	src/Dir_info.md
run_batch.py	Batch collection	src/	src/Dir_info.md
strategy_a.py	Batch collection	src/	src/Dir_info.md
strategy_b.py	Batch collection	src/	src/Dir_info.md
strategy_c.py	Batch collection	src/	src/Dir_info.md
extract_task_intent.py	Task analysis	src/task_analysis/	src/task_analysis/Dir_info.md
task_category_plots.py	Task analysis	src/task_analysis/	src/task_analysis/Dir_info.md
annotate_tasks.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
label_tool.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
cohen_kappa.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
tier_diagrams.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
rubric.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
annotator.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
gold_set.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
observed_cost.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
postprocess.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
reporting.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
agreement.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
tasks.py	Task annotation	src/task_annotation/	src/task_annotation/Dir_info.md
remove_errors.py	Decontamination	src/clean_contamination/	src/clean_contamination/Dir_info.md
sort_batch.py	Decontamination	src/clean_contamination/	src/clean_contamination/Dir_info.md
classify_tasks.py	Decontamination	src/clean_contamination/	src/clean_contamination/Dir_info.md

flag_episodes.py	Decontamination	src/clean_contamination/	src/clean_contamination/Dir_info.md
join_metadata.py	Decontamination	src/clean_contamination/	src/clean_contamination/Dir_info.md
build_transitions.py	Router training (Strategy C)	src/router_c/	src/router_c/Dir_info.md
train_router.py	Router training (Strategy C)	src/router_c/train/	src/router_c/Dir_info.md
compare_models.py	Router training (Strategy C)	src/router_c/model_analysis/	src/router_c/Dir_info.md
spending_scheme_config.py	Spending schemes (RQ3)	src/spending_schemes/	src/spending_schemes/Dir_info.md
allowance.py	Spending schemes (RQ3)	src/spending_schemes/	src/spending_schemes/Dir_info.md
agents.py	Spending schemes (RQ3)	src/spending_schemes/	src/spending_schemes/Dir_info.md
select_tasks.py	Spending schemes (RQ3)	src/spending_schemes/read_only_tasks/	src/spending_schemes/Dir_info.md
run_schemes.py	Spending schemes (RQ3)	src/spending_schemes/	src/spending_schemes/Dir_info.md
analyse_schemes.py	Spending schemes (RQ3)	src/spending_schemes/scheme_analysis/	src/spending_schemes/Dir_info.md
shared.py	Batch analysis	src/batch_analysis/	src/batch_analysis/Dir_info.md
analyse_a.py	Batch analysis	src/batch_analysis/	src/batch_analysis/Dir_info.md
analyse_b.py	Batch analysis	src/batch_analysis/	src/batch_analysis/Dir_info.md
analyse_c.py	Batch analysis	src/batch_analysis/	src/batch_analysis/Dir_info.md
compare_abc.py	Batch analysis	src/batch_analysis/	src/batch_analysis/Dir_info.md

B.5 Agent prompts and output schemas

This section reproduces every instruction text given to a model during the project, so that the claims made in Sections 3.4 and 3.5.3 can be checked against the actual prompts. Each text is passed to Pydantic AI as the agent's instructions (the system role of the OpenAI Responses API) and the model's reply is parsed into the typed schema shown in Table B.5.1; a reply that does not fit the schema is rejected by the library and retried. The per-step user message is built by wa_env.build_prompt() from the current observation (Listing B.5.1) and is identical for all three

strategies; Strategy B's Executor and Critic receive the same text with the Planner's sub-goal, and the Executor's proposed action, appended. Strategy C adds no prompt of its own: it calls Strategy A's agent or Strategy B's three agents exactly as defined here.

Table B.5.1 Typed output schemas (Pydantic models)

Schema (Strategy)	Field	Type	Description given to the model
AgentAction (A)	Reasoning	Str	One short sentence justifying the action
	Action	Str	A single BrowserGym action string
PlannerPlan (B)	Reasoning	Str	One sentence of analysis
	Plan	Str	The next sub-goal in plain English, not an action string
ExectutorAction (B)	Reasoning	Str	One sentence justifying the action
	Action	Str	A single BrowserGym action string
CriticVerdict (B)	Approve	Bool	True if the proposed action should be executed unchanged
	Reasoning	Str	One sentence explaining the verdict
	Revised_action	Str | None	A corrected BrowserGym action string, only when approve is False
TaskAnnotation (annotator)	Pages_to_traverse	Int 0-2	0 single page, 1 two-to-three pages, 2 four-plus or unbounded
	Retrieval_type	Int 0-2	0 read one value, 1 compare or filter a few, 2 aggregate or count over a set
	Interaction	Int 0-2	0 read-only, 1 one form or click sequence, 2 multi-step state change
	Target_locatability	Int 0-2	0 target named explicitly, 1 derivable from the page, 2 must be discovered by scanning
	confidence	Float 0-1	Annotator's confidence; below 0.7 when the task text is ambiguous about navigation

Listing B.5.1 The per step user message built by wa_env.build_prompt() shared by all strategies

```
Goal:
{task intent, e.g. "What is the top-1 best-selling product in 2022"}

URL: {current page URL}
LAST ACTION ERROR: {BrowserGym's last_action_error, or "none"}

ACTIONS YOU HAVE ALREADY TAKEN:
  {n}. {action}  --> {URL the action led to, truncated to 70 characters}
  ... (the last 8 actions; "none yet" on the first step)

PAGE (AXTree):
{accessibility tree of the current page, visible elements with a bid only}

--- Strategy B appends to the same text ---
Executor:                          Critic:

PLANNER SUB-GOAL:                  PLANNER SUB-GOAL:
{Planner's sub-goal}               {Planner's sub-goal}

                                   EXECUTOR PROPOSED ACTION:
                                   {Executor's action string}

--- Timeout substitutions for LAST ACTION ERROR (wa_env.build_prompt) ---
```

```
Same action twice at the same URL, both timed out:
  "this exact action has FAILED more than once and the page has not changed.
   It will not work. Choose a DIFFERENT element or a different approach -
   do not retry it."
Any other timeout:
  "previous action timed out waiting for the page to settle - it may have
   succeeded. Check the current page before retrying."
```

Strategy A's instruction block (Listing B.5.2, 3,268 characters, 817 tokens at the 4:1 estimate used by the budget check) contains two parts: the action vocabulary, which is the environment interface, and the rules for navigation and answering.

Listing B.5.2 Strategy A single-agent instructions (strategy_a.INSTRUCTIONS). The block above "Rules" is the action vocabulary reused by Strategy B

```
You are a web-navigation agent operating a website through BrowserGym.
You are given a task goal and the page's accessibility tree (AXTree). Each
element has a bracketed id such as [a31]. Return EXACTLY ONE action string,
using the id WITHOUT the brackets:

click('a31')                  - click an element
fill('b12', 'some text')      - type into a field
select_option('c4', 'Blue')   - choose a dropdown option
press('a31', 'Enter')         - press a key
scroll(0, 400)                - scroll the page
goto('http://...')            - navigate to a URL
go_back()                     - browser back
send_msg_to_user('answer')    - ANSWER an information-seeking task
noop()                        - do nothing

Rules:
- Only use ids that appear in the current AXTree.
- If the task asks a question, it is complete only once you call
  send_msg_to_user(...) with the answer.
- Keep reasoning to one sentence. Output one action only.
- Review ACTIONS YOU HAVE ALREADY TAKEN. Each line shows an action and the URL
  it led to. Element ids change on every page load, so the SAME destination can
  have a DIFFERENT id - judge repetition by the URL, not the id. If a URL
  appears repeatedly WITHOUT you reaching any new page in between, you are
  going in circles: try a different route or a different part of the page.
  Returning to a hub page to take a different branch is fine.
- Prefer navigating via links and scrolling over using site search boxes.
  Search endpoints are slow and often fail. Only use search if no navigational
  path is visible.
- If the page shows a server error (500, 502, 504) or fails to load, use
  go_back() and try a different route. Do NOT answer N/A because of a page
  error - N/A means the information genuinely does not exist.
- If the task is impossible to complete on this site (the data does not exist,
  or the site does not support the requested operation), send exactly:
  send_msg_to_user('N/A')
- Dropdowns come in two forms. Try select_option('378', 'games') first.
  If it errors with "Element is not a <select> element", the control is a
  custom widget: click it once, then look at the NEXT observation. A searchbox
  may appear - fill it with the option name to filter, then click the matching
  option. Do not retry select_option on such an element; it will never work.
- Some tasks only require you to REACH a page ("browse X", "search for Y",
  "go to Z"). These need no answer: once the correct page is loaded the task
  is complete. Do not click into individual items or navigate away - use
  noop() to stay on the page.

Answering with send_msg_to_user - the answer is graded by EXACT MATCH:
- Send ONLY the answer itself. No explanation, no preamble, no quotes,
  no sentence wrapping it.
  Correct: send_msg_to_user('Sprite Stasis Ball 65 cm')
  Wrong: send_msg_to_user('The top seller is Sprite Stasis Ball 65 cm')
- For a list of items, comma-separate them: send_msg_to_user('Alice, Bob')
- If nothing on the page satisfies the criteria, send exactly:
  send_msg_to_user('N/A')
- Numbers: send digits only, e.g. send_msg_to_user('0')
- Do NOT answer until you have navigated to and verified the specific
```

```
   information the task asks for. Answering early ends the episode and
   cannot be undone.
```

Strategy B's three role instructions (Listings B.5.3 to B.5.5) are built in strategy_b.py as f-strings that interpolate the action-vocabulary block of Listing B.5.2 unchanged, which the marked line in each listing denotes; a start-up assertion checks that the vocabulary is present in all three, the guard added after the second-version run described in Section 3.4.2. The three blocks total 8,212 characters, the 2,053-token instruction overhead charged to every Strategy B step (Section 3.3).

Listing B.5.3 Strategy B Planner Instructions (strategy_b.PLANNER_INSTRUCTIONS)

```
You are the PLANNER in a three-agent web-navigation pipeline. You decide what
should happen next. You do NOT produce actions and you do NOT answer the task.

{ACTION VOCABULARY - the first block of Listing B.5.2}


Given the goal, current page, and action history, output the next sub-goal in
one short sentence.

NAVIGATION
- Name the specific element or region, e.g. "open the user's profile via the
  username link", not "find the user".
- Element ids change on every page load. Judge repetition by the URL an action
  led to, not the id. If previous attempts have not changed the page, plan a
  DIFFERENT route.
- Returning to a hub page to take a different branch is legitimate; repeating
  the same branch is not.
- Prefer links and scrolling over search boxes; search endpoints often fail.
- If select_option was tried on an element without the page changing, it is a
  custom widget. Plan to click it open, then look for the option or a
  searchbox in the next observation.

INFORMATION TASKS
- If the answer is visible on this page, say "the answer is on this page:
  <value>" and the Executor will submit it.
- For counting, summing, or comparing across items, ensure ALL relevant items
  are visible first - scroll or paginate before answering.
- On admin pages, navigate to the correct section and apply filters BEFORE
  reading. Dashboard summaries may not match the required period or status.

NAVIGATION-ONLY TASKS
- If the task only requires reaching a page ("browse X", "search for Y") and
  you are on it, say "the target page has been reached; stay here". Do not
  click into individual items.
- Apply any required sort or filter BEFORE declaring the target reached.

STATE-CHANGE TASKS
- Calculate the target value ONCE from the ORIGINAL value on the page. After
  the action history shows the field was filled, plan to SAVE. Do NOT re-read
  and re-apply the modification - that compounds the change.
- Complete one field at a time; do not move on while a required field still
  shows its default.
- Once all fields are filled, the next sub-goal is "click the Save button".
  Identify it by label (Save, Submit, Post, Create), not position - a
  "Submit" link in the site header is navigation, not the form's submit.

CREATE TASKS
- Identify all required fields first, then fill them in order, then submit.
- For forum posts, select the forum FIRST and confirm it before title/body.
- For rating or review tasks, navigate to the product before looking for the
  review form.

BULK ACTIONS
- For "like all" / "dislike all" tasks, reach the page listing all items, then
  act on each sequentially, scrolling to reveal more.

DELETE TASKS
```

```
- Some admin views require checkbox selection then a bulk action - plan for
  that pattern rather than looking for a per-row delete button.

IMPOSSIBLE TASKS
- If you have checked and confirmed the task cannot be done on this site, say
  "this task is impossible on this site" and the Executor will submit N/A.

ERROR RECOVERY
- On a server error (500, 502, 504) or a page that fails to load, plan to go
  back and try a different route.
```

Listing B.5.4 Strategy B Executor instructions (strategy_b.EXECUTOR_INSTRUCTIONS)

```
You are the EXECUTOR in a three-agent web-navigation pipeline. A Planner has
given you one sub-goal. Translate it into exactly one action.

{ACTION VOCABULARY - the first block of Listing B.5.2}


- Only use element ids that appear in the current AXTree.
- Do exactly what the sub-goal asks. Do not pursue a different route.
- Use the exact value the sub-goal specifies. Do not recalculate or rephrase.

DROPDOWNS AND WIDGETS
- For standard dropdowns use select_option. If the sub-goal says to click a
  widget open or type into a searchbox, do that instead.

FORM FILLING
- Put multi-line text (descriptions, post bodies) in a single fill() call.
- Identify save/submit buttons by their label (Save, Submit, Post, Create).
  A "Submit" link in the site header navigates away - it is not the form's
  submit button.

ANSWERING
- Graded by EXACT MATCH. Send ONLY the value, no explanation or sentence.
      Correct:   send_msg_to_user('Sprite Stasis Ball 65 cm')
      Wrong:     send_msg_to_user('The top seller is Sprite Stasis Ball 65 cm')
- Numbers as digits only. Lists comma-separated. Include currency symbols if
  shown on the page. If nothing matches, or the task is impossible, send 'N/A'.

NAVIGATION
- If the sub-goal says the page has been reached, use noop().
- On a page error (500, 502, 504), use go_back().
```

Listing B.5.5 Strategy B Critic instructions (strategy_b.CRITIC_INSTRUCTIONS)

```
You are the CRITIC in a three-agent web-navigation pipeline. You are shown the
goal, current page, the Planner's sub-goal and the Executor's proposed action.
Decide whether it should be executed.

{ACTION VOCABULARY - the first block of Listing B.5.2}


FIRST: confirm the element id in the proposed action actually appears in the
current AXTree. Do not claim an element is present without checking -
approving actions on non-existent elements is the most common failure.

Approve unless there is a concrete problem:

ELEMENT PROBLEMS
- The element id is not in the current AXTree.
- The action uses syntax outside the vocabulary above.
- select_option or fill is proposed on an element where the same action type
  already appears in the action history at this URL without the page changing.

STATE-CHANGE PROBLEMS
- The same field is being filled with a DIFFERENT value than a previous fill
  on that element - the agent is re-applying a modification that was already
  made. Suggest saving instead.
- A form is being submitted with a required field still empty or at default.
```

```
- The action clicks a header navigation link instead of the form's submit.

ANSWER PROBLEMS
- The answer is malformed: wrapped in a sentence, explained, or not an exact
  match to what the task asks for.
- The answer is submitted before the information was verified on the page, or
  counted from a partial list.

NAVIGATION PROBLEMS
- The action does not serve the stated sub-goal.
- The target page was reached but the action navigates away from it.
- The page shows a server error but the action is not go_back().

If you reject, supply revised_action drawn from the current AXTree. Do not
reject because you would have chosen a different route - only when the action
is wrong.
```

The high-effort variants used by the RQ3 front-loaded and reactive schemes (spending_schemes/agents.py) use the identical texts and differ only in the reasoning-effort setting.

Listing B.5.6 is the instruction given to the difficulty annotator (task_annotation/rubric.py); the 20 gold exemplars are appended to it at run time as additional scored examples, and the observed-cost hint, when present, is added to each task's user message.

Listing B.5.6 Difficulty annotator instructions (task_annotation/rubric.ANNOTATOR_INSTRUCTIONS)

```
You score WebArena web-agent tasks on four difficulty dimensions for a study of token budgets.

PAGES TO TRAVERSE - how many distinct pages must be visited?
    0 = everything needed is on the starting page
    1 = two or three pages
    2 = four or more, or an unbounded search across pages

RETRIEVAL TYPE - what must be done with the information?
    0 = read a single stated value
    1 = compare or filter a small number of items
    2 = aggregate, count, or reason over a set of items

INTERACTION - what must be done to the site?
    0 = read-only; nothing on the site changes
    1 = one form submission or click sequence
    2 = a multi-step state change (create, edit, delete, configure)

TARGET LOCATABILITY - how hard is the target to find?
    0 = named explicitly in the task, e.g. "the Sprite Stasis Ball"
    1 = derivable from what is on the page
    2 = must be discovered by scanning or searching

You measure the task's STRUCTURAL DEMANDS, not an agent's capability. Judge on
the number of distinct interactions and the reasoning depth required, not on
sentence length or surface wording.

You may also be shown OBSERVED COST: how many steps and distinct pages a
baseline agent needed. Use it to calibrate PAGES TO TRAVERSE, which
is difficult to judge from text alone. Do NOT use it to infer the other three
dimensions, and note that an agent may wander - observed pages is an upper
bound on what the task requires, not the exact number.

Give an honest confidence. Use below 0.7 when the task text leaves the amount
of navigation genuinely ambiguous.
```

Appendix C – Dataset

C.1 Episode JSONL Schema

Table 3.2 Termination reasons recorded for every episode and the failure class.

Reason	Activates when	Class
Success	WebArena's evaluator returns reward 1	Success
Env_terminated	An answer is submitted and judged wrong	Wrong answer
Budget_would_exceed	The pre-step estimate (Equation 3.1) would exceed the cap	Budget exhausted
Safety_token_cap	The running total reaches the cap after a call	Budget exhausted
Budget_exhausted_mid_step	The cap is reached between B's three calls	Budget exhausted
Navigation_cycle	The same action is issued from the same page three times	Stuck (guard)
Repeated_action_failure	Four consecutive actions return an error	Stuck (guard)
Router_stop	Strategy C's stop head ends the episode (Section 3.4.3)	Router stop

C.2 Datasets inventory

Table C.2.1 Dataset inventory: every dataset the project produced or used, the module responsible for it and where it is used. Locations are in Table C.5.1.

Dataset	Module (producer)	What it is	Used by
strategy_a.jsonl, strategy_b.jsonl, strategy_c.jsonl	Batch collection (run_batch.py)	Raw main-grid logs: one JSON record per episode (2,850 per strategy) with per-call step logs	Decontamination pipeline; A's file also gives the annotator its observed-cost hint
strategy_c_routing.jsonl	Batch collection (strategy_c.py)	One row per C episode: the router's chosen mode, the stop probability at every step and the stop step.	Router-behaviour checks (Section 4.3.3)
pilot_a.jsonl, pilot_b.jsonl	Batch collection	The first pilot runs of Strategies A and B	Development provenance only; not in the results
strategy_b_v1/v2/v3_instructions.jsonl	Batch collection	Strategy B episodes collected under instruction	Prompt provenance for Section 3.4.2; not in the headline results

		versions v1–v3, with the instruction texts alongside	
test.raw.json	WebArena (input)	WebArena's raw task configurations for all 812 tasks	extract_task_intent.py
task_intents.json	Task analysis (extract_task_intent.py)	Intent and category for every kept task (475), with per-site and per-category summaries	classify_tasks.py; the annotator; reporting
task_risk_levels.jsonl	Decontamination (classify_tasks.py)	The contamination-risk level of every task: read_only, idempotent or non_idempotent (Table 3.3)	flag_episodes.py, join_metadata.py, select_tasks.py
gold_set_50.jsonl, gold_set_50.csv	Task annotation (annotate_tasks.py, label_tool.py)	The 50-task gold set: blank rows, then my hand labels entered through the CSV	Annotator calibration (20 exemplars) and κ validation (30 held out)
tasks_annotated_run2.jsonl	Task annotation (annotate_tasks.py)	The LLM annotator's raw scores for every task (run 2)	cohen_kappa.py; the final labels
kappa_run2.json	Task annotation (cohen_kappa.py)	Hand-label against annotator agreement per dimension; the source of Table 3.5	Report (Section 3.5.3)
task_metadata.jsonl	Task annotation (annotate_tasks.py)	The final difficulty labels per task: four dimension scores, total, tier and label source	join_metadata.py; every analysis; select_tasks.py
episodes_flagged.jsonl, episodes_flagged_c.jsonl	Decontamination (flag_episodes.py)	All episodes with the first-success rule applied and	join_metadata.py

		use_for_reward set	
final_episodes.jsonl	Decontamination (join_metadata.py)	The combined decontaminated A+B episodes with difficulty and risk metadata	Router training (build_transitions.py)
final_episodes_a.jsonl, final_episodes_b.jsonl, final_episodes_c.jsonl	Decontamination (join_metadata.py)	The per-strategy decontaminated datasets behind every Chapter 4 table	analyse_a/b/c.py, compare_abc.py; analyse_schemes.py (pay-as-you-go reference)
mode_pairs.jsonl	Router training (build_transitions.py)	One row per (task, budget) with A's and B's success and tokens side by side (2,850 pairs, 2,560 eligible)	train_router.py (mode head)
transitions.jsonl	Router training (build_transitions.py)	One row per step of every A/B episode plus a final stop row (23,801 transitions)	train_router.py (stop head)
router.pt, metrics.json, oof_predictions.jsonl (models 1–3)	Router training (train_router.py)	Each trained router: both heads' weights, feature normalisation and stop threshold; its metrics; its held-out predictions	strategy_c.py loads model_2; compare_models.py
read_only_tasks.jsonl	Spending schemes (select_tasks.py)	The fixed RQ3 sample: 100 read-only tasks stratified by tier (33/33/34)	run_schemes.py, analyse_schemes.py
schemes.jsonl	Spending schemes (run_schemes.py)	The 900 RQ3 episodes: 3 strategies × 3 schemes × 100 tasks at the 32k budget	analyse_schemes.py

C.3 Task classification listing

Table C.3.1 WebArena task scope by site. 475 single site tasks on the 3 hosted sites form the task population.

Site	Single site tasks	In scope
Shopping (OneStopShop)	187	Yes
Shopping Admin (Magneto CMS)	182	Yes
Forum (Postmill)	106	Yes
GitLab	180	No – not hosted
Map	109	No – not hosted
Multi-site tasks	48	No
Total	812	475 used

C.4 Spending-Scheme configuration

Table C.4.1 Four spending schemes at the 32k budget. Allowances are capped at the budget remaining.

Scheme	Step 0	Later steps	Reasoning Effort
Pay as you go	All remaining	All remaining	Low
Even	6,400	6,400	Low
Front loaded	12,800 (40% of the budget)	4,800	Hight at step 0, then low
Reactive	6,400	9,600 after an error or an unchanged page; 4,800 after progress	High when stuck, otherwise low

C.5 Access/Location

All datasets are publicly available in the project repository. The paths in Table C.5.1 are relative to the repository root. The main datasets that answered the research questions are in the Additional Material/Datasets directory (available in both).

Table C.5.1 Location of each dataset in the public repository.

Dataset	Repository location
strategy_a.jsonl	data/raw/6_budgets_ALL_tasks_batch/batch_Strategy_A/
strategy_b.jsonl	data/raw/6_budgets_ALL_tasks_batch/batch_Strategy_B/
strategy_c.jsonl, strategy_c_routing.jsonl	data/raw/6_budgets_ALL_tasks_batch/batch_Strategy_C/
pilot_a.jsonl, pilot_b.jsonl	data/raw/pilot_batch_outputs/small_batch_test_Strategy_A/ and ..._Strategy_B/
strategy_b_v1/v2/v3_instructions.jsonl, v1–v3 instruction texts	data/raw/strategy_B_instruction_versions/
gold_set_50.jsonl, gold_set_50.csv, tasks_annotated_run2.jsonl, kappa_run2.json	data/raw/annotation_tests/run2/ (the first annotation run is in data/raw/annotation_tests/run1/)
test.raw.json, task_intents.json, task_risk_levels.jsonl, read_only_tasks.jsonl	data/processed/task_list/
task_metadata.jsonl	data/processed/final_annotation_difficulty_tiers/
episodes_flagged.jsonl, final_episodes.jsonl	data/processed/6_budgets_ALL_tasks_decontaminated_batch/
final_episodes_a.jsonl / _b.jsonl / _c.jsonl (and episodes_flagged_c.jsonl)	data/processed/6_budgets_ALL_tasks_decontaminated_batch/batch_strategy_A/, .../batch_strategy_B/, .../batch_strategy_C/
mode_pairs.jsonl, transitions.jsonl	data/processed/router_c_models/
router.pt, metrics.json, oof_predictions.jsonl	data/processed/router_c_models/model_1/, model_2/ (the deployed router), model_3/
schemes.jsonl	data/processed/schemes_output/

Appendix D – Generative AI Use

D.1 Code generation – Prompt log

Listing D.1.1 label_tool.py

```
Script: label_tool.py
Model: Claude Opus 5
Prompt: add a function to label_tool.py that checks the completed CSV hand labelling gold set file
and compares it to the original unedited gold set JSONL to check for any inconsistencies
Lines/Output: 206-250
What I've Learnt: It surfaced a check I hadn't thought to apply to myself: my hand labels had to
agree with each other before they could serve as ground truth.
Mistakes Made: None
```

Listing D.1.2 classify_tasks.py

```
Script: classify_tasks.py
Model: Claude Opus 5
Prompt: Write a self-test function which includes these 32 selected tasks [attached txt file of
tasks]
Lines/Output: 263-345
What I've Learnt: This allowed me to develop similar functions in other scripts to check the
validity such as in flag_episodes.py
Mistakes Made: None
```

Listing D.1.3 shared.py

```
Script: shared.py
Model: Claude Opus 5
Prompt: Write a function or 2 if needed that first calculates at least a 95% confidence range for a
success rate, given the number of successes and the total number of attempts. It needs to work well
when the success rate is very low or zero. And I also need to test whether 1 strategy is genuinely
more successful than another when both were run on the same tasks. It should only look at the tasks
where one strategy succeeded and the other failed.
Lines/Output: 75-84 and 90-97
What I've Learnt: I learnt that the usual "±1.96 × standard error" interval fails at low success
rates. It can drop below 0% and collapses to zero width when there are no successes, which is common
at 2k–4k budgets. The Wilson interval stays within 0–1 and stays sensible near zero. From McNemar I
learnt that when strategies run on the same tasks, only the tasks where they disagree carry any
evidence about which is better. Treating the results as two independent samples would ignore that
pairing and give the wrong p-values.
Mistakes Made: None
```

Listing D.1.4 compare_abc.py

```
Script: compare_abc.py
Model: Claude Opus 5
Prompt: My analysis script compare_abc.py compares three web-agent strategies (A, B and C) using
exact McNemar tests on paired (task, budget) cells. It runs the comparison for each strategy pair (A
vs B, A vs C, B vs C), both pooled across budgets and within each difficulty tier (Easy, Medium,
Hard). Because this involves many tests at once, apply a correction method.
Lines/Output: 123-150 & 291-313
What I've Learnt: I learnt about the multiple comparisons problem. Running many tests at the 5%
level makes it likely that at least one comes out "significant" by chance alone. Dividing the
threshold by the number of tests (0.05/3 for the pooled pairs, 0.05/9 within tiers) means only
differences too large to be luck are claimed. I also learnt that Bonferroni is conservative, so a
borderline result like A vs C (p = 0.065) should be reported as no detectable difference rather than
as evidence that the two are equal.
Mistakes Made: None
```

Listing D.1.5 analyse_schemes.py

```
Script: analyse_schemes.py
Model: Claude Opus 5
```

```
Prompt: write two functions that compare my results task by task. The first compares each pair of
spending schemes within each strategy, testing both success and tokens spent. The second compares
the strategies against each other under the same scheme.
Lines/Output: 114-147 & 150-171
What I've Learnt: I learnt that success and cost need different paired tests. So 1st for success as
whether yes/no per task, so it uses McNemar on the tasks where the two schemes disagree. And 2nd for
tokens which are continious, so they use the Wilcoxon signed rank test on each tasks difference,
which doesnt assume a normal distribution.
Mistakes Made: None
```

Appendix E – Project Management and Ethics

E.1 Research Ethics Review Form (CSREC)

Research Ethics Review Form: BSc, MSc and MA Projects

Computer Science Research Ethics Committee (CSREC)

http://www.city.ac.uk/department-computer-science/research-ethics

Undergraduate and postgraduate students undertaking their final project in the Department of Computer Science are required to consider the ethics of their project work and to ensure that it complies with research ethics guidelines. In some cases, a project will need approval from an ethics committee before it can proceed. Usually, but not always, this will be because the student is involving other people ("participants") in the project.

In order to ensure that appropriate consideration is given to ethical issues, all students must complete this form and attach it to their project proposal document. There are two parts:

PART A: Ethics Checklist. All students must complete this part.
The checklist identifies whether the project requires ethical approval and, if so, where to apply for approval.

PART B: Ethics Proportionate Review Form. Students who have answered "no" to all questions in A1, A2 and A3 and "yes" to question 4 in A4 in the ethics checklist must complete this part. The project supervisor has delegated authority to provide approval in such cases that are considered to involve MINIMAL risk. The approval may be ***provisional*** – *identifying the planned research as* likely to involve MINIMAL RISK. In such cases you must additionally seek ***full approval*** from the supervisor as the project progresses and details are established. ***Full approval*** must be acquired in writing, before beginning the planned research.

A.1 If you answer YES to any of the questions in this block, you must apply to an appropriate external ethics committee for approval and log this approval as an External Application through Research Ethics Online - https://ethics.city.ac.uk/	*Delete as appropriate*

1.1	Does your research require approval from the National Research Ethics Service (NRES)? *e.g. because you are recruiting current NHS patients or staff?* *If you are unsure try - https://www.hra.nhs.uk/approvals-amendments/what-approvals-do-i-need/*	**NO**
1.2	Will you recruit participants who fall under the auspices of the Mental Capacity Act? *Such research needs to be approved by an external ethics committee such as NRES or the Social Care Research Ethics Committee - http://www.scie.org.uk/research/ethics-committee/*	**NO**
1.3	Will you recruit any participants who are currently under the auspices of the Criminal Justice System, for example, but not limited to, people on remand, prisoners and those on probation? *Such research needs to be authorised by the ethics approval system of the National Offender Management Service.*	**NO**
A.2 If you answer YES to any of the questions in this block, then unless you are applying to an external ethics committee, you must apply for approval from the Senate Research Ethics Committee (SREC) through Research Ethics Online - https://ethics.city.ac.uk/		*Delete as appropriate*
2.1	Does your research involve participants who are unable to give informed consent? *For example, but not limited to, people who may have a degree of learning disability or mental health problem, that means they are unable to make an informed decision on their own behalf.*	**NO**
2.2	Is there a risk that your research might lead to disclosures from participants concerning their involvement in illegal activities?	**NO**
2.3	Is there a risk that obscene and or illegal material may need to be accessed for your research study (including online content and other material)?	**NO**

2.4	Does your project involve participants disclosing information about special category or sensitive subjects? *For example, but not limited to: racial or ethnic origin; political opinions; religious beliefs; trade union membership; physical or mental health; sexual life; criminal offences and proceedings*	**NO**
2.5	Does your research involve you travelling to another country outside of the UK, where the Foreign & Commonwealth Office has issued a travel warning that affects the area in which you will study? *Please check the latest guidance from the FCO - http://www.fco.gov.uk/en/*	**NO**
2.6	Does your research involve invasive or intrusive procedures? *These may include, but are not limited to, electrical stimulation, heat, cold or bruising.*	**NO**
2.7	Does your research involve animals?	**NO**

2.8	Does your research involve the administration of drugs, placebos or other substances to study participants?	**NO**
A.3 If you answer YES to any of the questions in this block, then unless you are applying to an external ethics committee or the SREC, you must apply for approval from the Computer Science Research Ethics Committee (CSREC) through Research Ethics Online - https://ethics.city.ac.uk/ **Depending on the level of risk associated with your application, it may be referred to the Senate Research Ethics Committee.**		*Delete as appropriate*
3.1	Does your research involve participants who are under the age of 18?	**NO**
3.2	Does your research involve adults who are vulnerable because of their social, psychological or medical circumstances (vulnerable adults)? *This includes adults with cognitive and / or learning disabilities, adults with physical disabilities and older people.*	**NO**
3.3	Are participants recruited because they are staff or students of City, University of London? *For example, students studying on a particular course or module.* *If yes, then approval is also required from the Head of Department or Programme Director.*	**NO**
3.4	Does your research involve intentional deception of participants?	**NO**
3.5	Does your research involve participants taking part without their informed consent?	**NO**
3.5	Is the risk posed to participants greater than that in normal working life?	**NO**
3.7	Is the risk posed to you, the researcher(s), greater than that in normal working life?	**NO**
A.4 If you answer YES to the following question and your answers to all other questions in sections A1, A2 and A3 are NO, then your project is deemed to be of MINIMAL RISK. **If this is the case, then you can apply for approval through your supervisor under PROPORTIONATE REVIEW. You do so by completing PART B of this form.** **If you have answered NO to all questions on this form, then your project does not require ethical approval. You should submit and retain this form as evidence of this.**		*Delete as appropriate*
4	Does your project involve human participants or their identifiable personal data? *For example, as interviewees, respondents to a survey or participants in testing.*	**NO**

E.2 Work plan: planned against actual

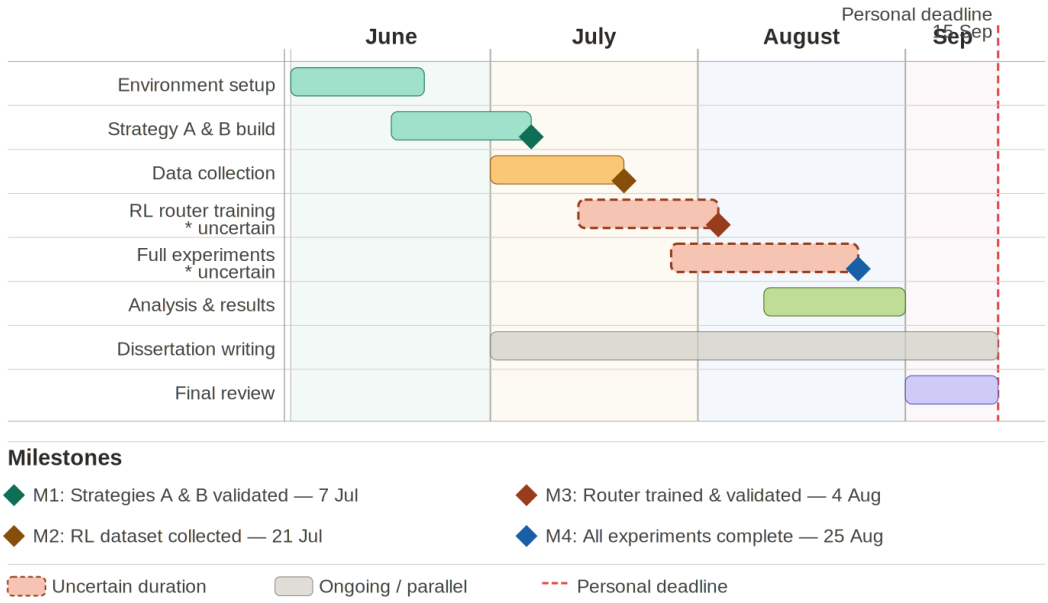


Figure E.2.1 Planned Gantt Chart

Table E.2.1 Actual dates of each project phase, taken from project journal (Appendix E.5). Figure E.2.1 is the planned schedule from the original proposal

Phase	Actual dates (2026)
Proposal revision and environment setup (AWS instance, WebArena containers)	29 Jun – 20 Jul
Harness and strategy implementation (wa_env.py, Strategies A and B)	10 – 28 Jul
Strategy A main-grid collection	29 Jul – 12 Aug
Strategy B main-grid collection	7 – 12 Aug
Decontamination pipeline and error-handling scripts	10 – 13 Aug
Difficulty annotation, gold set and validation	18 – 24 Sep
Router training and model selection (Strategy C)	24 – 27 Sep
Strategy C main-grid collection and decontamination	25 – 29 Sep
Spending-scheme runs (RQ3)	28 – 29 Sep
Analysis and report writing	22 Sep – 14 Oct

E.3 Risk register review

#	Risk	Type	Likelihood	Impact	Mitigation	Recovery
R1	**Time overrun on uncertain phases (RL training, full experiments)**	Project mgmt	High	High	Buffer weeks allocated in timeline (E.2). Modular implementation allows partial	Reduce scope — fewer budget levels or subset of WebArena tasks.

					completion. Uncertain tasks flagged early and prioritised.	Prioritise RQ1 and RQ2 over RQ3 if necessary.
R2	**API cost overrun — total spend exceeds $30-60 budget**	Technical	Medium	High	Real-time token accounting wrapper tracks cumulative API spend. Pilot experiments on small task subset before full runs. Hard spend cap set in OpenAI dashboard.	Switch to cheaper model tier. Reduce episodes per condition. Use cached completions where available.
R3	**WebArena / BrowserGym environment instability or setup failure**	Technical	Medium	High	Pin all Docker image and dependency versions. Run validation suite before full experiment runs. Environment setup allocated dedicated time in June.	Re-run affected episodes from checkpointed state. Escalate to BrowserGym GitHub issues. Revert to pinned stable release.
R4	**RL router fails to converge — policy collapses to single routing choice**	Technical	Medium	High	Entropy regularisation (alpha in J(pi)) prevents premature convergence. Monitor training loss and routing diversity. Validate on held-out episode set.	Fall back to rule-based heuristic router. Report as limitation and still compare Strategy A vs B.
R5	**GPT-4o-mini API deprecation,**	Infrastructure	Low	High	Monitor OpenAI API changelog	Switch model mid-project.

	pricing change, or rate limiting				throughout project. Claude Haiku identified as equivalent-cost alternative. Prompts designed to be model agnostic.	Re-run pilot to verify comparable baseline performance before continuing.
R6	**Insufficient or low-quality training data for offline RL router**	Research	Medium	Medium	Collect episodes across all 4 budget levels and difficulty tiers to maximise diversity. Record step-level features for richer router conditioning.	Reduce MLP router complexity to prevent overfitting. Apply data augmentation. Report data limitations explicitly.
R7	**Inconclusive results — no significant difference between strategies**	Research	Low	Medium	Bootstrap CIs ensure statistical sensitivity. Effect sizes reported. Difficulty-stratified analysis may surface differences masked in aggregate results.	Report null result as valid empirical contribution. Broaden analysis to error profiles and failure modes.
R8	**Data or experiment log loss — results or model weights deleted/corrupted**	Infrastructure	Low	High	Daily automated backups to GitHub and Google Drive. Checkpoint saves after every 50	Restore from most recent checkpoint. Re-run only episodes completed

					episodes. Model weights versioned separately.	after last backup.
R9	**Researcher unavailability due to illness or unforeseen circumstances**	Project mgmt	Low	High	Personal deadline (15 Sep) provides 9-day buffer before submission (24 Sep). Regular supervisor meetings to flag early delays.	Communicate with supervisor immediately. Reprioritise to core objectives. University extenuating circumstances process available if required.
R10	**Reproducibility failure — results cannot be independently verified**	Research	Low	Medium	All dependency versions pinned and documented. Random seeds fixed and recorded. Full environment specification published with dissertation code.	Re-run on pinned Docker environment before submission. Document any non-determinism explicitly in methodology.

Shading key: High = red, Medium = amber, Low = green

E.4 Record of supervisory contact

05/08/2026 – Email regarding project methodology changes from preliminary proposal.

22/09/2026 16:00 to 16:30 – AI Usage meeting.

E.5 Project Journal

This is in the additional material directory **root/docs/Additional Material/Journal.docx** (The journal has not been pushed to the repository due to containing sensitive information and is submitted through the additional materials submission.)

Appendix F – Original Proposal

The original project proposal, as approved, is reproduced unchanged on the following pages. Its work plan is the planned schedule that Appendix E.2 compares against the actual dates.

Token Budged Orchestration Strategies for Web Agents

Student Name: Thomas Beard | 220008104
Supervisor: Eduardo Alonso | **Meeting Schedule:** TBD

1 Introduction

Large Language Model (LLM) agents are increasingly deployed to complete real world web tasks automatically such as filling forms, navigating e-commerce sites & querying databases and is a rapidly growing area of applied AI research (Zhou et al., 2023; Xu, F. et al. 2024). Multi-agent systems, in which several specialised LLM agents collaborate to complete a task, have demonstrated improved performance over single-agent approaches (Wang et al., 2024b). However, they introduce a critical practical constraint: token cost compounds with every agent call. As Chen et al (2026) documents, weekly token processing volume across major agent platforms grew nearly 68-fold between December 2024 and March 2026 (figure A.2), driven precisely by the iterative, multi-step nature of agentic workflows. In multi-agent settings specifically, deeper interaction chains trigger more LLM calls, lengthen inter-agent contexts, and add serialisation overhead which is structurally coupling performance to cost in ways that single agent evaluations do not capture (Cai, Ning and Liu, 2025).

However, most existing frameworks are designed to maximise accuracy under expanded or unrestricted compute budgets, and rarely incorporate mechanisms for budget control (Lu et al., 2026b). Without structured resource management, agents frequently exhaust token limits on redundant steps, with blindly allocating more resources often producing only marginal accuracy gains and diminishing returns (Lu et al., 2026b). The gap between the token cost incurred by multi-agent systems and the absence of principled mechanisms to control it is the central problem this project addresses.

This project addresses that gap through a controlled empirical comparison of three orchestration strategies: a single-agent baseline (Strategy A), a fixed multi-agent pipeline (Strategy B), and a dynamically routed multi-agent system using offline reinforcement learning (Strategy C). All three are evaluated under fixed global token budgets on WebArena (Zhou et al., 2023), a benchmark of 812 realistic web tasks across five self-hosted websites (figure A.1). The research question (table A.1) is whether dynamic, budget-aware token routing can match or exceed the task success of fixed pipelines while consuming fewer tokens and is a question with direct implications for the cost to viability of deploying web agents at scale.

The following objectives (which is further discussed in sections 3 & 4) will be completed by the project deadline:

Implement and validate three orchestration strategies (A, B, and C) within the BrowserGym/WebArena environment, with a token accounting wrapper enforcing hard per-episode budget caps.

Collect a labelled dataset of heuristic policy episodes across four token budget levels ({2,000; 4,000; 8,000; 16,000} tokens) to serve as training data for the Strategy C offline RL router.

Train and evaluate a lightweight MLP router using offline reinforcement learning with entropy regularisation, comparing its routing behaviour against Strategies A and B at matched budgets.

Produce empirical token performance curves and difficulty stratified breakdowns addressing all three research questions (table A.1).

Submit dissertation, reporting all findings.

This project will produce four meaningful and concrete outputs: (1) A trained offline RL routing model (Strategy C); (2) An annotated dataset of agent episodes across budget conditions; (3) empirical cost performance curves and statistical comparisons across all three strategies.

The cost performance curves (3) are built from two primary per-episode measurements: task success rate (recorded as a binary label using WebArena's built in evaluation scripts) and token consumption (recorded as total input + output tokens across all LLM calls within an episode). Furthermore, the accounted metrics will also follow: (i) Token efficiency (success rate / mean tokens consumed); (ii) cost curves plotting success rate against each budget level per strategy and finally (iii) difficulty stratified breakdowns disaggregating both metrics by WebArena's Easy/Medium/Hard task tiers. And a cross-strategy comparison at matched budgets will use confidence intervals alongside probability values.

Regarding the four budget levels ({2,0000; 4,000; 8,000; 16,000} tokens per episode) is standard practice for producing evaluation points capable of revealing non-linear scaling behaviour and diminishing return thresholds (Lu et al., 2026b). So, for 2,000 tokens, this represents a constrained floor sufficient for simple tasks, whereas 16,000 tokens provide a comfortable ceiling for strategy A and remains meaningfully constrained for strategy B's three agent pipeline.

2 Literature Review

2.1 The Web Agent Setting: WebArena as Benchmark

LLM agents that autonomously complete web tasks have advanced rapidly, but evaluation has lagged. Early benchmarks used simplified environments that did not reflect real web complexity (Shi et al., 2017). WebArena (Zhou et al., 2023) addresses this with 812 tasks across five self-hosted websites, evaluated programmatically against functional success conditions (figure 1). Its key value for this project is reproducibility as the Docker based environment eliminates the confounding variability of live websites, ensuring differences in success rate are attributable to orchestration design rather than environmental drift. Even GPT-4 achieved only 14.41% success, confirming WebArena as a genuine challenge space where architectural differences between strategies produce measurable outcomes.

2.2 Multi-Agent Collaboration and the Fixed Pipeline (Strategy B)

Multi-agent systems improve on single-agent performance by decomposing tasks across specialised roles. Wang et al., (2024b) formalises this in the Mixture of Agents (MoA) architecture, where each agent at layer *i* conditions on all outputs from layer *i-1*:

[1] A{i,j} = LLM(prompt, {A{i-1,1}, …, A{i-1,k}})

[1] A{i,j} is a specific agent, so A{2,1} means "the first agent in the second layer." {A{i-1,1}, ..., A{i-1,k}} is the set of outputs from all agents in the previous layer. So before generating its response, each agent is shown everything the agents before it produced. MoA achieved state of the art performance on AlpacaEval 2.0 (65.1%), surpassing GPT-4 Omni using only open-source models. Strategy B operationalises MoA's layered principle as a fixed sequential pipeline: Planner → Executor → Critic. CodeAgents (Yang et al., 2025b) further motivates structured decomposition, demonstrating 55–87% input token reductions through codified planning. However, the critical limitation of any fixed pipeline is budget-blindness: all three agents are invoked every step regardless of task complexity or remaining budget, costing approximately 3–4x the tokens of a single-agent step (figure A.3).

2.3 Token Cost as a Structural Barrier

The token cost problem in multi-agent systems is a structural barrier to deployment at scale. As mentioned in the introduction section, Chen et al (2026) documents that that weekly token processing volume across major agent platforms grew 68-fold between December 2024 and March 2026, driven by coordination overhead in multi-agent workflows. Salim & Khatoonabadi (2026c) provide empirical grounding finding that iterative agent collaboration stages account for most of the token usage, with input tokens (accumulated from prior agent outputs passed as context) resulting in 53.9% of total consumption. Zhang et al., (2024b) further demonstrated that a significant fraction of inter-agent communication is redundant, so removing non-essential messages achieved 28–73% token reduction with no meaningful accuracy loss across six benchmarks.

2.4 Budget-Aware Orchestration (Strategy C)

BAMAS (Yang et al., 2025c) is the most directly related prior work, framing multi-agent construction as a constrained optimisation problem and using an RL policy to select collaboration topologies under budget constraints. It demonstrated up to 86% cost reduction at comparable performance. This project extends BAMAS Algorithm 1 from per-task topology selection to per-step routing decisions (choosing from {Planner, Executor, Critic, Stop} at each action step) conditioned on remaining budget, step count and task difficulty. The router is a lightweight MLP trained using offline RL with the objective:

[2] $J(\pi) = \mathbb{E}_\pi [R(\tau)] + \alpha \cdot H(\pi)$

[2] $J(\pi)$ is the overall score the router is trying to maximise. $\mathbb{E}_\pi [R(\tau)]$ is the expected reward the router receives by following its current policy (π) across episodes (τ). A high reward means the episode succeeded cheapy, and a low reward means it failed or was token expensive. $\alpha \cdot H(\pi)$ is the entropy where (H) measures how varied the routers decisions are (i.e. a router that always picks “Executor” has low entropy, whereas one that spreads decisions across the different agents has high entropy). The coefficient (α) controls how much this bonus is weighted. This means the router is also rewarded for maintaining decisional variety.

2.5 Evaluation Methodology in Agent Research

Most frameworks report accuracy without accounting for the cost at which that accuracy was achieved (Mohammadi et al., 2025d). The token efficiency metric used in this project follows the cost of pass framing introduced by Wang et al. (2024c), which quantifies performance per unit of compute.

3 Methodology

3.1 Experimental Design

This study employs a controlled quantitative design in which three independent variables are systematically varied: strategy type (A, B, C), token budget ({2,000; 4,000; 8,000; 16,000} tokens per episode), and task difficulty (Easy/Medium/Hard). All three strategies are evaluated on the same WebArena tasks using the same underlying model (GPT-4o-mini) within the BrowserGym environment, ensuring that differences in measured outcomes are attributable to orchestration design rather than confounding factors. The primary dependent variables are task success rate and token consumption per episode, from which three derived metrics are computed: token efficiency ($\eta = SR / \mu(T)$), cost curves, and difficulty-stratified breakdowns.

3.2 Strategy Implementations

All three strategies share a common token-accounting wrapper that intercepts every LLM API call, records input and output tokens, and enforces a hard per-episode budget cap by terminating the episode when the cumulative token count reaches the assigned budget level. This ensures budget enforcement is uniform and strategy-agnostic.

Strategy A implements a single GPT-4o-mini agent following the baseline framework from WebArena (Zhou et al., 2023). The agent receives the task prompt and web observation at each step, produces an action, and the budget is decremented accordingly, establishing the cost-performance floor.

Strategy B implements the fixed Planner → Executor → Critic pipeline (Figure A.3), grounded in the MoA architecture (Wang et al., 2024b). At each step, the Planner receives the task and current observation and produces a structured plan; the Executor receives the plan and produces an action; the Critic evaluates the action. Each agent receives all prior outputs as context, following equation [1]: $A_{i,j} = LLM(prompt, \{A_{i-1,1}, \ldots, A_{i-1,k}\})$. The token budget is checked after the Critic each step; if exhausted, the episode terminates. This incurs 3–4× the per-step token cost of Strategy A.

Strategy C implements the MLP router trained using offline RL. At each step, the router receives a feature vector comprising: task embedding, remaining token budget (B), step count, episode progress flag, and difficulty tier. It outputs a routing decision from {Planner, Executor, Critic, Stop}, invoking only the selected agent for that step. The router is trained to maximise the objective from equation [2]: $J(\pi) = \mathbb{E}\pi[R(\tau)] + \alpha \cdot H(\pi)$, where R(τ) encodes both task success and token cost, and the entropy term prevents collapse to a single routing strategy. The Stop action allows the router to conserve budget when continued steps are unlikely to yield success.

3.3 Data Collection and Router Training

Training data for the Strategy C router is collected by running Strategies A and B across all four budget levels on a representative subset of WebArena tasks, generating labelled episode trajectories. Each trajectory step is recorded as a (feature vector, action taken, outcome) tuple. The offline RL MLP is then trained on this dataset using policy gradient with entropy regularisation. The router is fixed after training and is not updated during evaluation, ensuring a fair comparison at matched budgets.

3.4 Analysis

Per-episode measurements (binary success label, total tokens consumed) are aggregated to compute: (i) success rate per condition; (ii) token efficiency $\eta = SR / \mu(T)$; (iii) cost curves plotting success rate against budget level for each strategy; and (iv) difficulty-stratified breakdowns by WebArena's Easy/Medium/Hard task tiers. Cross-strategy comparisons at matched budget levels use bootstrap confidence intervals (10,000 resamples) given the binary nature of the success metric, with effect sizes reported alongside probability values. A key methodological limitation is distributional shift in the offline RL training data: the router is trained on episodes generated by Strategies A and B, which may not cover the full range of states encountered during evaluation. This is a known challenge in offline RL (Yang et al., 2025c) and is mitigated by the entropy regularisation term.

3.5 Ethics

This project raises no ethical concerns. All research involves automated software agents interacting with a sandboxed, self-hosted benchmark environment. No human participants, personal data, deception, or sensitive materials are involved. The CSREC Ethics Review Form has been completed and is included as Appendix B, with all questions answered No, confirming that no ethical approval is required.

4 Timeline

The Gantt chart below outlines all project phases from June to September 2025, with key dependencies and milestones identified. Strategy C's RL router cannot be trained until the heuristic episode dataset is collected (M2), and full experiments cannot begin until the router is trained and validated (M3). Dissertation writing runs in parallel throughout, with a dedicated review period in the final two weeks before the personal deadline of 15 September 2025.

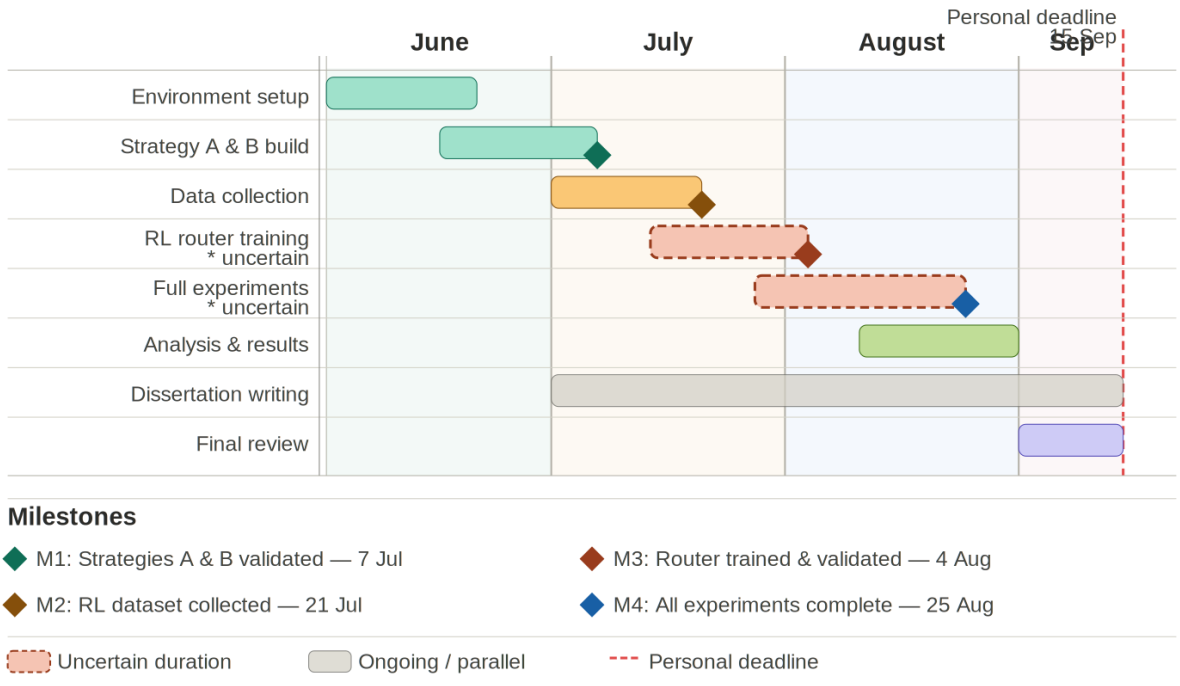


Figure 4: Project work plan — Token-Budgeted Orchestration Strategies for Web Agents

5 Risks

As this project involves no human participants (confirmed by the CSREC Ethics Review Form, Appendix B), risks to third parties are minimal. The register below covers technical, research, infrastructure, and project management risks. Likelihood and impact are rated High / Medium / Low and shaded accordingly.

#	Risk	Type	Likelihood	Impact	Mitigation	Recovery
R1	Time overrun on uncertain phases (RL training, full experiments)	Project mgmt	High	High	Buffer weeks allocated in timeline (Fig. 4). Modular implementation allows partial completion. Uncertain tasks flagged early and prioritised.	Reduce scope — fewer budget levels or subset of WebArena tasks. Prioritise RQ1 and RQ2 over RQ3 if necessary.
R2	API cost overrun — total spend exceeds $30-60 budget	Technical	Medium	High	Real-time token accounting wrapper tracks cumulative API spend. Pilot experiments on small task subset before full runs. Hard spend cap set in OpenAI dashboard.	Switch to cheaper model tier. Reduce episodes per condition. Use cached completions where available.
R3	WebArena / BrowserGym environment instability or setup failure	Technical	Medium	High	Pin all Docker image and dependency versions. Run validation suite before full experiment runs. Environment setup allocated dedicated time in June.	Re-run affected episodes from checkpointed state. Escalate to BrowserGym GitHub issues. Revert to pinned stable release.
R4	RL router fails to converge — policy collapses to single routing choice	Technical	Medium	High	Entropy regularisation (alpha in J(pi)) prevents premature convergence. Monitor training loss and routing diversity. Validate on held-out episode set.	Fall back to rule-based heuristic router. Report as limitation and still compare Strategy A vs B.
R5	GPT-4o-mini API deprecation, pricing change, or rate limiting	Infrastructure	Low	High	Monitor OpenAI API changelog throughout project. Claude Haiku identified as equivalent-cost alternative. Prompts designed to be model-agnostic.	Switch model mid-project. Re-run pilot to verify comparable baseline performance before continuing.
R6	Insufficient or low-quality training data for offline RL router	Research	Medium	Medium	Collect episodes across all 4 budget levels and difficulty tiers to maximise diversity. Record step-level features for richer router conditioning.	Reduce MLP router complexity to prevent overfitting. Apply data augmentation. Report data limitations explicitly.

R7	Inconclusive results — no significant difference between strategies	Research	Low	Medium	Bootstrap CIs ensure statistical sensitivity. Effect sizes reported. Difficulty-stratified analysis may surface differences masked in aggregate results.	Report null result as valid empirical contribution. Broaden analysis to error profiles and failure modes.
R8	Data or experiment log loss — results or model weights deleted/corrupted	Infrastructure	Low	High	Daily automated backups to GitHub and Google Drive. Checkpoint saves after every 50 episodes. Model weights versioned separately.	Restore from most recent checkpoint. Re-run only episodes completed after last backup.
R9	Researcher unavailability due to illness or unforeseen circumstances	Project mgmt	Low	High	Personal deadline (15 Sep) provides 9-day buffer before submission (24 Sep). Regular supervisor meetings to flag early delays.	Communicate with supervisor immediately. Reprioritise to core objectives. University extenuating circumstances process available if required.
R10	Reproducibility failure — results cannot be independently verified	Research	Low	Medium	All dependency versions pinned and documented. Random seeds fixed and recorded. Full environment specification published with dissertation code.	Re-run on pinned Docker environment before submission. Document any non-determinism explicitly in methodology.

Shading key: High = red (#FFD9D9) Medium = amber (#FFF3CD) Low = green (#D4EDDA)

6 References

Zhou et al., (2023) Zhou, S., Xu, F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U. and Neubig, G. (n.d.). WebArena: A Realistic Web Environment for Building Autonomous Agents. [online] Available at: https://webarena.dev/static/paper.pdf

Xu, F. et al. (2024). TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. arXiv:2412.14161 [online] Available at: https://arxiv.org/pdf/2412.14161

Wang et al., (2024b) Arxiv.org. (2022). Mixture-of-Agents Enhances Large Language Model Capabilities. [online] Available at: https://arxiv.org/html/2406.04692v1

Chen, Y., Chen, J., He, C., Li, Y., Ji, Y., Wu, Y., Yang, D., Diao, L., Shou, L., Zhang, H., Li, H. and Chen, G. (2026). Token Economics for LLM Agents: A Dual-View Study from Computing and Economics. [online] arXiv.org. Available at: https://arxiv.org/abs/2605.09104

Cai, S., Ning, Y. and Liu, H. (2025). AgentBalance: Backbone-then-Topology Design for Cost-Effective Multi-Agent Systems under Budget Constraints. arXiv:2512.11426 [online] arXiv.org. Available at: https://arxiv.org/abs/2512.11426

Li, Y., Deng, W., Li, J. and Li, X. (2026b). Spend Less, Reason Better: Budget-Aware Value Tree Search for LLM Agents. arXiv:2603.12634 [online] arXiv.org. Available at: https://arxiv.org/abs/2603.12634

Shi, T., Andrej Karpathy, Fan, L., Hernandez, J. and Liang, P. (2017). World of Bits: An Open-Domain Platform for Web-Based Agents. PMLR, [online] pp.3135–3144. Available at: https://proceedings.mlr.press/v70/shi17a.html

Yang, B., He, X., Gao, H., Cao, Y., Li, X. and Hsu, D. (2025b). CodeAgents: A Token-Efficient Framework for Codified Multi-Agent Reasoning in LLMs. [online] arXiv.org. Available at: https://arxiv.org/abs/2507.03254

Salim, M., Latendresse, J., Khatoonabadi, S. and Shihab, E. (2026c). Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering. [online] arXiv.org. Available at: https://arxiv.org/abs/2601.14470

Zhang, G., Yue, Y., Li, Z., Yun, S., Wan, G., Wang, K., Cheng, D., Yu, J.X. and Chen, T. (2024b). Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. [online] arXiv.org. Available at: https://arxiv.org/abs/2410.02506.

Yang, L., Luo, J., Liu, X., Lou, Y. and Chen, Z. (2025c). BAMAS: Structuring Budget-Aware Multi-Agent Systems. [online] arXiv.org. Available at: https://arxiv.org/abs/2511.21572

Mohammadi, M., Li, Y., Lo, J. and Yip, W. (2025d). Evaluation and Benchmarking of LLM Agents: A Survey. pp.6129–6139. Available at: https://doi.org/10.1145/3711896.3736570

Wang, J., Jain, S., Zhang, D., Ray, B., Kumar, V. and Athiwaratkun, B. (2024c). Reasoning in Token Economies: Budget-Aware Evaluation of LLM Reasoning Strategies. [online] arXiv.org. Available at: https://arxiv.org/abs/2406.06461

Appendix A – Figures & Tables

Figure 1: (Zhou et al., 2023) WebArena Structure

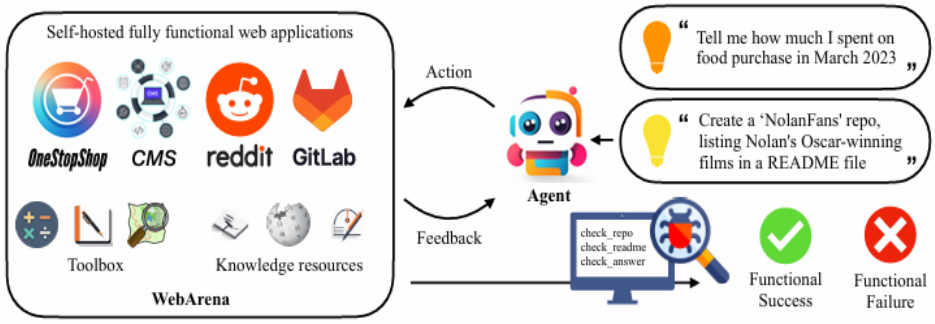


Figure 2: (Chen et al., 2026) Token Usage 2024 - 2026

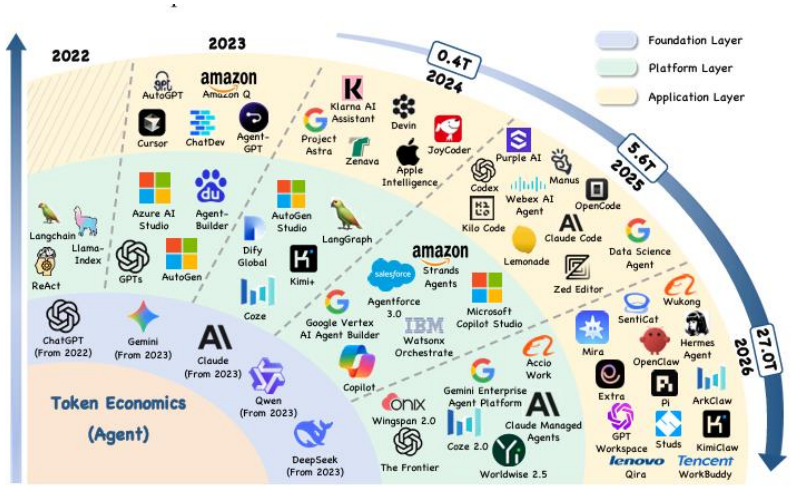


Table 1: Research Questions

No.	Research Question
RQ1	How does task success rate, token efficiency and error profile vary as a function of a fixed global token budget when comparing the 3 orchestration strategies on WebArena?
RQ2	How do the three strategies differ in sensitivity to task difficulty (Easy / Medium / Hard) in terms of success rate and failure modes across WebArena tasks?
RQ3	Given the same global token budget, how do different intra-episode allocation schemes (front-loaded, even-distribution, reactive) affect success rate and token efficiency across all three strategies?

Figure 3: Strategies Architecture Diagram (Drawn on Draw.io)

